Final Report

CMSC-495-7381-2232-CTPnCS

Group 1

Peter Koropey, Benjamin Young, Min-Hsuan Yu, Anishka Forbes

University of Maryland: Global Campus

CMSC 495: Current Trends and Projects in Computer Science

Professor Majid Shaalan

May 9, 2023

Contents

Overview

Machine learning (ML) has become an essential tool in many areas of research and industry, making it increasingly important for students to learn about the various algorithms and techniques used in the field. To this end, our group has developed an application utilizing the Python programming language that allows users to train and test original implementations of the K-Nearest Neighbors and Random Forest ML models using default or customized parameters. The software includes a graphical user interface (GUI) to facilitate use of the two ML algorithms, written as original implementations by the group members. The primary objective of the software is to support educational purposes by enabling users to gain a deeper understanding of diverse ML models and how adjusting parameters can influence prediction results.

Methodology:

The software utilizes the MNIST handwritten digits database to train and test various ML models. The GUI allows users to select the desired model and customize various parameters. Users can also choose to train the model using the default parameters. The software includes two ML algorithms implemented from scratch by the group members: a K-Nearest Neighbors algorithm and a Random Forest algorithm.

Results:

The output generated by the software is presented using a comprehensive graphical display with accompanying explanations. The GUI displays the predicted results, as well as the receiver operating characteristic **(**ROC) curve graph. The software allows users to compare the performance of different models and parameters, gaining a deeper understanding of the advantages and limitations of each model.

Purpose:

The software developed by our group provides a valuable tool for students to learn about various ML models and how adjusting parameters can influence prediction results. The GUI and accompanying visualizations make it easy for users to compare and analyze the performance of different models, gaining insights into their strengths and limitations. The two ML algorithms implemented from scratch by the group members add an extra dimension to the educational experience, allowing users to understand the fundamental principles behind these algorithms. Overall, our software provides a valuable resource for anyone interested in learning about ML models and their applications.

Project Plan

1. Define project goals and requirements:
    - Define the scope of the project.
    - Identify the target audience.
    - Determine the required features of the software.
2. Develop a project plan:
    - Define milestones and deadlines.
    - Assign roles and responsibilities.
    - Plan for quality assurance and testing.
3. Collect and prepare data:
    - Obtain the MNIST handwritten digits database.
    - Clean and preprocess the data.
4. Develop the software:
    - Design and implement the GUI.
    - Implement the K-Nearest Neighbors and Random Forest algorithms.
    - Integrate the algorithms into the GUI.
    - Implement the option to customize model parameters.
    - Develop the output display and accompanying visualizations.
5. Test the software:
    - Conduct unit tests and functional tests.
    - Test the software with a sample of users.
    - Fix any bugs or issues identified during testing.
6. Deploy the software:

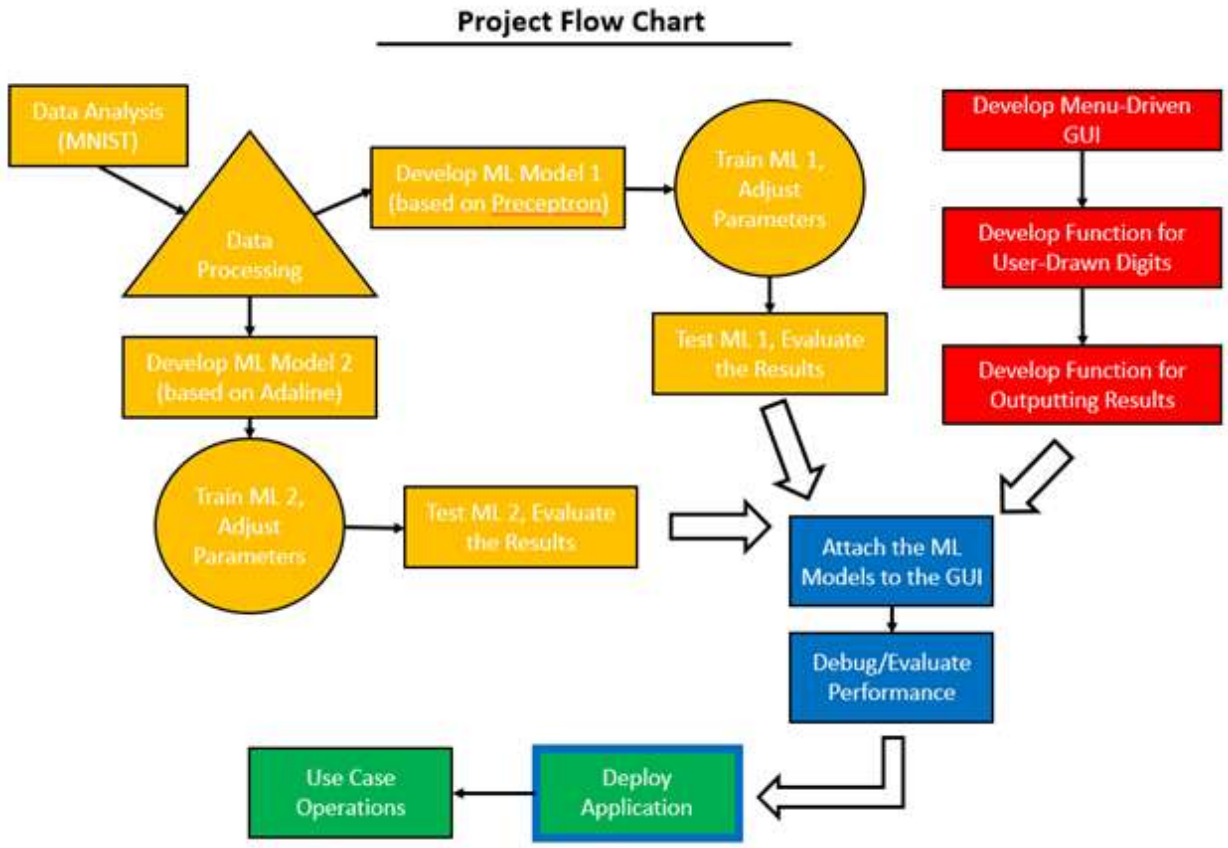
- Package the software for distribution.
- Provide instructions for installation and usage.

Milestone and Responsibilities:

- Milestone 1: Data Acquisition and preprocessing - all of the team members
- Milestone 2: Model Development and Training -
  - Min-Hsuan Yu (K-Nearest Neighbors algorithm)
  - Benjamin Young (Random Forest Algorithm)
- Milestone 3: Model Evaluation and Optimization -
  - Min-Hsuan Yu (K-Nearest Neighbors algorithm, developing evaluation and ROC curve plotting methods)
  - Benjamin Young (Random Forest Algorithm)
- Milestone 4: Development of User Interface
  - Part 1A: GUI Design and Layout - Anishka Forbes
  - Part 1: Model and Parameter Selection - Peter Koropey
- Milestone 5: Development of User Interface
  - Part 2: Classifying a User-Drawn Digit - Peter Koropey
- Milestone 6: Final Testing and Deployment - all of the team members
  - Testing: Anishka Forbes and Benjamin Young
  - Deployment: Min-Hsuan Yu and Peter Koropey
- Milestone 7: Documentation - all of the team members

NOTE: Milestones 3 and 5 to be developed concurrently. Milestones 4 and 6 to be developed concurrently, after Milestones 3 and 5 are reached.

Flow Chart 1: Project Development



Requirement Specification

- The software must be able to read the MNIST handwritten digits database and train various machine-learning models using the data.
- The graphical user interface must allow users to easily draw a digit, select the desired model and customize various parameters.
- The software must include two machine learning algorithms implemented from scratch by the group members: a K-Nearest Neighbors algorithm and a Random Forest Algorithm.
- The Software must be able to use the selected Machine Learning Model to classify a black-and-white digit drawn by a user on an MS-Paint style canvas.
- The output generated by the software must be presented using a comprehensive graphical display with accompanying explanations.
- The GUI must display the predicted results and pertinent metrics, including Precision, Recall, F1 Score, and a Receiver Operator Characteristic graph for the selected machine learning model.
- The software must generate visualizations of the results, including an ROC graph and a means of
- The software must allow users to compare the performance of different models and parameters, gaining a deeper understanding of the advantages and limitations of each model.

System Specification

- Contourpy: v. 1.0.7
- Cycler: v. 0.11.0
- Fonttools: v. 4.39.2
- importlib-resources: v. 5.12.0
- joblib: v. 1.2.0
- kiwisolver: v. 1.4.4
- matplotlil: v. 3.7.1
- numpy: v. 1.24.2
- packaging: v. 23.0
- pandas: v. 1.5.3
- Pillow: v. 9.4.0
- Pyparsing: v. 3.0.9
- python-dateutil: v. 2.8.2
- pytz: v. 2022.7.1
- scikit-learn: v. 1.2.2
- scipy: v. 1.10.1
- six: v. 1.16.0
- threadpoolctl: v. 3.1.0
- zipp: v. 3.15.0
- annoy: v. 1.0.3
- tkinter: v. 8.6

Most of these are already installed with Python. Those which are not can be installed using the “pip” utility. For information on how to use Pip, please see: https://pip.pypa.io/en/stable/getting-started/

User's Guide

The user's guide provides step-by-step instructions for setting up and running the MNIST app. Before running the application, make sure to have Python v. 3.11.3 installed along with all libraries listed in the previous section. Follow the steps below to set up and run the application:

1. Create a Virtual Python Environment to run the application by following the instructions provided at this link: https://python.land/virtual-environments/virtualenv.
2. Download the .zip file from https://github.com/papafranco69/MNIST_app/archive/refs/heads/main.zip.
3. Extract the ZIP file to your desired directory within the virtual environment.
4. Open Command Prompt (CMD).
5. Navigate to the virtual environment.
6. Run the program by entering "python Run_MNIST_App.py" in the command prompt. On a Windows PC, you can also double-click the "Run_MNIST_App.py" file to run the program.

Once the application is running, you can select, train, and test different machine learning models. The following steps explain how to do this:

1. Select a machine learning model using the drop-down selector located at the top left of the Main Menu.
    a. Available Machine Learning Models are K-Nearest Neighbor ("knn") and Random Forest. Refer to the "Available Machine Learning Models" section for more information.
2. To change the selected machine learning model's parameters, select the parameter and input a new value by either clicking or typing.
3. To adjust the value for partitioning the dataset into training and testing subsets, drag the slider across the desired value.
    a. This value represents the percentage of elements in the original dataset that will belong to the training subset.
    b. The "Random State Seed" value adjusts which elements are randomly selected for inclusion in the training subset but does not affect the size of the training subset.
4. Press the "Train ML Model" button at the top right of the window to train the selected machine learning model with the selected parameters.

a. Note that a message box will pop up to inform the user when training is complete, and it may take a few minutes, depending on the training subset size and selected model.

b. When training is complete, the trained model and its parameters will be displayed in the upper center of the window.

5. Press the "Test ML Model" at the top right to test the trained ML model's capabilities on the testing subset.

    a. Note that a message box will pop up to inform the user when testing is complete, and it may take a few minutes, depending on the testing subset size and selected model.

    b. When testing is complete, the console at the center-left will display values for the trained model's Precision, Recall, and F1 Score.

    c. An ROC chart for the trained model will be displayed in the center. The ROC chart can be displayed again by selecting "Graph" at the top left and clicking "ROC curve."

6. Press the "Draw Your own Digit!" button in the center left to bring up a canvas for drawing and submitting a numerical digit for classification.

    a. Note that a machine learning model must first be trained. Click and drag on the canvas to draw a digit.

    b. Press "Clear Canvas" at the bottom to erase the drawing.

    c. Press "Submit your Digit!" at the center-left to have your trained ML model classify your drawing.

    d. When the digit is classified, a message box will pop up with the classification, asking whether the machine learning model accurately predicted the user's drawing. See Figure 5. The accuracy of classification will be affected by the parameters of the trained ML model.

7. To exit the program, press "File" at the top left and select "Quit."

Adjusting Parameters

You can adjust the machine learning model parameters in the left side of the window. The parameters vary based on the machine learning model you choose. You can find more information about the available machine learning models in the "Available Machine Learning Model" section.

Different parameters require different types of inputs, such as text, drop-down boxes, checkmarks, or sliders. If the parameter requires text input, make sure to enter a positive integer.

Some parameters are constant for each machine learning model. These include:

- Enable Preprocessing: Check this option to enable preprocessing on the MNIST dataset.
- Random State Seed: This determines which MNIST dataset elements will be assigned (pseudo-randomly) to the training and testing subsets.
- Percentage of Dataset Used for Training: This determines the proportion of the MNIST dataset that will be assigned to the training subset. The remaining portion will be assigned to the testing subset.

For example, the default value of 75% assigns approximately 52,500 elements of the MNIST dataset for training the selected machine learning model. The remaining 17,500 elements will be used for testing the machine learning model.

Partitioning the Dataset

Before training a Machine Learning model on a dataset, it is necessary to divide the dataset into two subsets: a training subset and a testing subset. The model is first trained on the training subset and then evaluated on the testing subset to ensure its accuracy in classifying new data.

Increasing the size of the training subset relative to the testing subset can improve the accuracy of the model, but it will also increase the time required to train the model. However, if the training subset is too large, the model may become overfitted, meaning it will have difficulty in classifying new data beyond the training and testing datasets.

To ensure that the model performs well in the real world, it is recommended to strike a balance between the size of the training and testing subsets. During the drawing of a digit, the user may observe if the model is underperforming in accuracy of classification. The size of the training and testing subsets can be adjusted in the Parameters section of the user interface.

User-Drawn Digits

The User-Drawn Digit function allows you to draw a single-digit number using your mouse and pass it to a trained machine learning model for classification. This helps you to test how well the model performs outside of the established MNIST dataset.

To use the User-Drawn Digit function:

1. Make sure that a machine learning model is selected and trained with the desired parameters. If a machine learning model is not trained, an error message will appear.
2. Click on “Draw Your Own Digit” located at the center-right.
3. Click and drag the left mouse button on the canvas in the center of the window to draw your digit.

    a. Use the “Clear Canvas” button at the bottom left to erase your drawing.

4. Click the “Submit Your Digit!” button at the center-left to classify your drawn digit.
5. The MNIST App will convert your drawing into a form of data compatible with the MNIST dataset, and it will be classified as if it were another element in that dataset.
6. A pop-up window will display the results of the classification. You can inform the program whether or not the drawing was correct.

Note: This program was designed to classify single-digit numbers, based on the data in the MNIST dataset. Drawing two-digit (or greater) numbers, multiple numbers, or non-numbers will not work.

Receiver Operator Characteristic (ROC) Graphs

ROC graphs demonstrate a machine learning model’s effectiveness by plotting the classified True Positives (TP) on Y-axis against False Positives (FP) on the X-axis.

In binary classification problems, we often evaluate the performance of a machine learning model by looking at the number of true positives and false positives it produces.

A true positive occurs when the model correctly identifies a positive instance as positive. For example, if we are building a model to detect whether an email is spam or not, a true positive occurs when the model correctly identifies a spam email as spam.

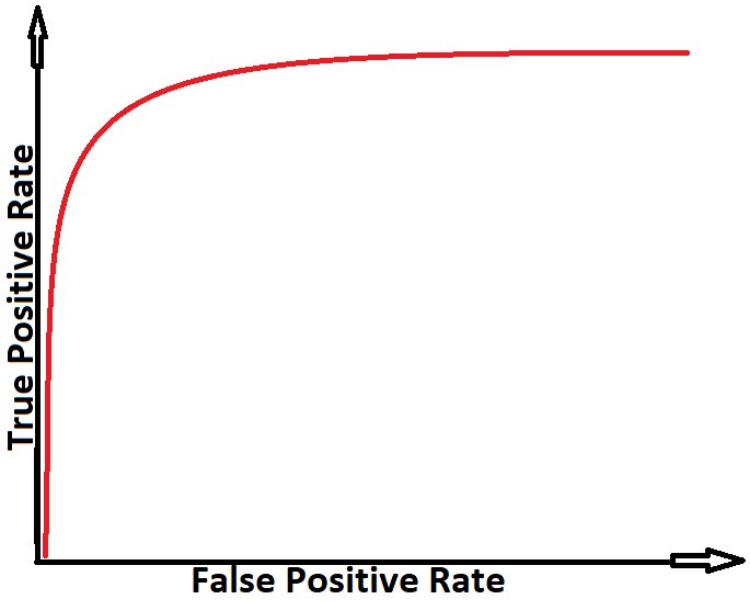
On the other hand, a false positive occurs when the model incorrectly identifies a negative instance as positive. Using the same email spam detection example, a false positive occurs when the model incorrectly identifies a non-spam email as spam.

Negatives (N) are samples that are classified incorrectly. For example, if a value is classified as a “1” when it is in fact a “0”, this is a negative for classifying 0 (and an FP for classifying 1).

The MNIST dataset contains 10 classes: the integers 0-9. If, for example, a machine learning model classifies every single value as a 0, then the 0-classifier has a TP rate of 100% (since it is correctly classifying every 0). However, this model also has an FP rate of 100% (since it is incorrectly classifying every digit 1-9 as a 0).

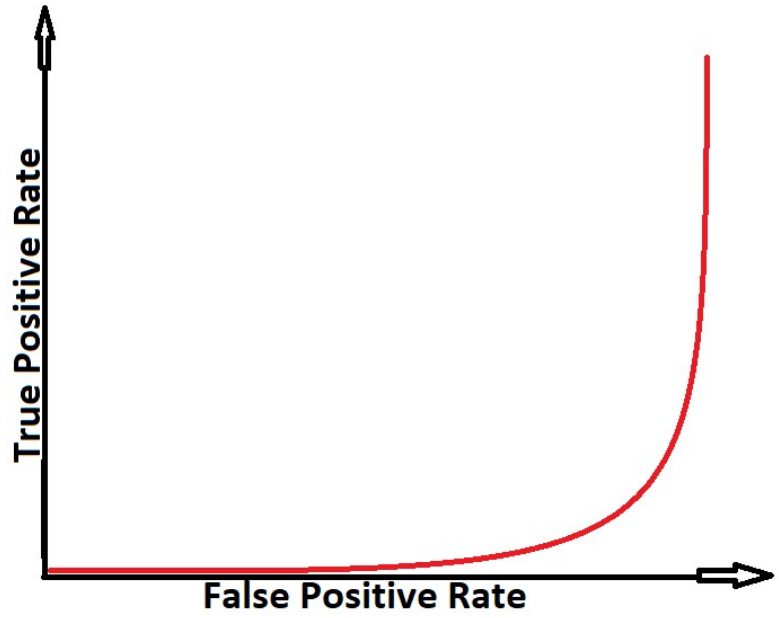
An effective machine-learning model will display high TP rates and low FP rates. This means it is correctly classifying each data sample. An example of the ROC chart of an effective machine-learning model is displayed below:

Chart 1: ROC of an Effective ML Model



An ineffective machine learning model will display high TP rates only for high FP rates. This means it is classifying many data samples incorrectly for each sample it classifies correctly. An example of a ROC chart of an ineffective machine-learning model is displayed below:

Chart 2: ROC of an Ineffective ML model



A perfect classifier would have a TPR of 1 and an FPR of 0, resulting in a point at the top left corner of the ROC curve. A random classifier, on the other hand, would produce a diagonal line from the bottom left to the top right corner, with an area under the curve (AUC) of 0.5.

In general, a ROC curve with a higher AUC indicates better performance of the model, as it indicates that the model is better at distinguishing between the positive and negative instances. An AUC of 1.0 indicates a perfect classifier, while an AUC of 0.5 indicates a random classifier.

ROC charts are generated automatically when testing a machine learning model. They can be recalled later (if hidden by the digit-drawing canvas) by pressing “Graph” at the top left corner and selecting “ROC Chart”.

Precision, Recall, F1-Score

The MNIST App also displays the Precision, Recall, and F1 Score for a selected ML model after testing. These can be read in the console box in the center-right of the main window. Definitions for each metric are provided below:

Precision: True Positives / (True Positives + False Positive)

Recall: True Positives / Total Sample Values of that Class

F1 Score: 2 * (Precision * Recall) / (Precision + Recall)

Precision demonstrates how many elements classified to a certain class-value were correct.

Recall demonstrates how many elements of a certain class-value were classified correctly.

Machine Learning Models

This section includes a brief description of the available machine learning models in this program along with their parameters. Parameters are the settings you can adjust to control the behavior of the model.

K-Nearest Neighbors (KNN) Parameters:

- K (number of neighbors): Positive Integer. Default Value: 5
- Weight Function: Selection (Uniform or Distance). Default Value: Uniform

KNN is a non-parametric learning model that can operate in supervised or unsupervised varieties. In this program, we use supervised KNN models, which means that the machine knows the classes of data for the training subset. KNN works by calculating the distances between each element in the testing subset and all elements of the training subset. Then, the model finds the "K" number of classified elements with the least distance to the element under examination. The examined

element is then grouped into the class with the greatest number of instances among the "K" neighbors with the least distance.

Random Forest Parameters:

- Decision Depth: Positive Integer. Default Value: 4
    - Note that increasing this value will increase the time the model takes to train.
- Number of Estimators: Positive Integer. Default Value: 10

Random Forest is an ensemble machine learning model developed from the iterative use of multiple decision trees to decrease variance in the decision tree learning model by aggregating the prediction of each of the individual trees. This helps reduce the overfitting of the model to the training data.

Test Plan and Results

Overview

Below are the test results of each of the test cases from the Test Plan. Due to changes in the development process there have been a few adjustments. For example, instead of testing input validation for data partitioning, the developer changed the input to a slider to prevent anything other than a percentage from passing through the application.

Objective

The test plan for this software project will cover the following aspects:

GUI Testing:

- Ensure that all GUI components and controls are responsive and functional.
- Test the GUI on different operating systems and screen resolutions to ensure that it is compatible and consistent.
- Ensure that the GUI displays the output in a comprehensive graphical display as intended.
- Test the ability to adjust machine learning model parameters through the GUI.
- Test the ability to draw a digit for classification and select a machine-learning **model through the GUI.**

Machine Learning Model Testing:

- Test the return results of the machine learning models on the MNIST handwritten digits dataset.
- Ensure that the machine learning models can be trained and tested using default or customized parameters.
- Test the ability to save and load customized parameters trained machine learning models.
- Test the ability to integrate the machine learning models with the GUI to display the output in a comprehensive graphical display with accompanying explanations.

Error Handling: The software should be able to handle and report errors and exceptions encountered during usage.

Testing Methodology

GUI Testing Methodology:

The GUI has numerous functions which must operate successfully. The functions are summarized below:

- Machine Learning Model Selection: The GUI shall include a dropdown combo-box for selecting available machine learning models.
- Machine Learning Parameter Input: For each available machine learning model, the GUI shall include functions for inputting values for parameters for each model. The available parameters vary from model to model.
- Partitioning the Dataset: The GUI shall include a feature which allows the user to partition the dataset into training and testing subsets.
- Displaying Testing Subset Results: The GUI shall include functions for displaying machine learning model testing results as tables or charts.
- Drawing a Digit: The GUI shall include a feature allowing the user to draw a numeric digit on a canvas. The digit will then be classified by the user's trained machine learning model.
- Centering and Resizing the User-Drawn Digit: The GUI application must include a feature which centers the user-drawn digit about its "center of mass" and resizes it to 28x28 pixels. This is required to make the digit consistent with the data in the MNIST dataset used for this program.

Each of these functions will be tested as part of this plan.

Scope

This test plan provides guidance for testing the GUI functionality for the MNIST App. For this plan to be fully effective, the module for the GUI must be able to interface with modules for the machine learning models (the testing of which is described later in this report).

The functions listed in the section above will each be tested in the course of this plan. Test cases for model selection, partitioning, and parameter input will be reproducible. User-drawn image classification testing is not perfectly reproducible because it is unlikely users will be able to draw the exact same digit multiple times. However, the integer matrix derived from a hand drawn digit is reproducible, and the centering and resizing of that matrix can be replicated. A modified function which takes an image (bitmap) file as input and outputs a text file matrix will be used to ensure precision of the image-matrix conversion.

Risks and Constraints

The primary constraint in testing the GUI is the imperfect reproducibility of testing the user-drawn digit function. This requires decomposing the composite functions into their parts, which can then be tested individually as part of other modules. If all the parts are working correctly according to the test cases described, and the composite works correctly according to the test case described, then it can be assumed the module as a whole is functional.

Generating Test Data:

Unit testing on GUI functions for ML model selection and parameter input requires data only in the form of valid and invalid inputs to ensure input error checking is correct. This is provided in the test cases.

Unit testing on GUI functions for drawing a digit requires data in the form of bitmap image files and text-file matrices and arrays, the nature of which is described in these test cases. The bitmap image files are easily generated in MS-Paint. The matrices and arrays are generated by methods within the ImgMatConv.py file, called via the ImgMatTest.py in the "test" folder (see Test Cases 17-21).

Integration testing on the GUI, Integration testing on the modules for digit-drawing and classification, requires the completion of all classes, including Machine Learning classes. Testing will then commence.

Testing Strategy:

Tests for basic GUI functionality (menu selection, dropdown box selection, button pressing) will be performed atomically. These will be grouped into more complex test-cases for testing learning model selection, parameter input, dataset partitioning, and learning model training. The test results from learning model testing will then attempt to replicate learning model testing results based on given parameters.

The digit-drawing function will be composed of atomic "click-and-drag" mouse actions for drawing a number. For testing the validity of centering, resizing, and classification, an integer matrix corresponding to a specific hand-drawn digit will be used, and provided to the trained machine learning model with given parameters for classification. The centered and resized matrix will be compared to an expected output. Classification precision testing will be based on replicated machine learning model parameters. Unfortunately, there is no guarantee that the machine learning model will classify a digit correctly – however, the result, correct or incorrect, must be reproducible.

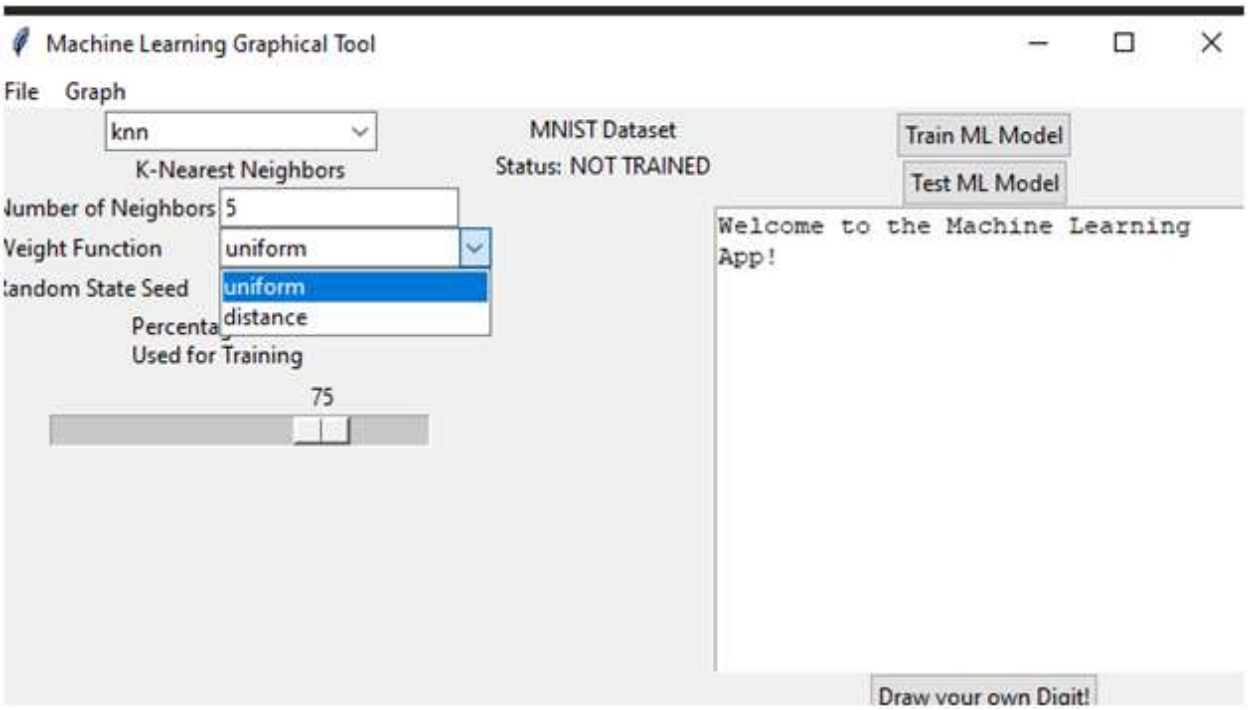
Machine Learning Model Testing Methodology:

1. Unit Testing:
    a. Test individual functions and methods of the machine learning models to ensure they perform as expected.
    b. Model Robustness: Test the models with different levels of noise in the input dataset to evaluate their robustness to data corruption.
2. Scalability Testing:
    a. Train and test the models with different training/testing split ratios to evaluate their performance.
    b. Use previously known input-output pairs as a baseline to compare new results against and ensure consistency.
    c. Measure the performance of the models using precision, recall, and f1 score.
3. Integration Testing:
    a. Test the integration of machine learning models with the GUI to ensure they work together seamlessly.
    b. Test the interaction between different components of the machine learning models to ensure they produce consistent results.

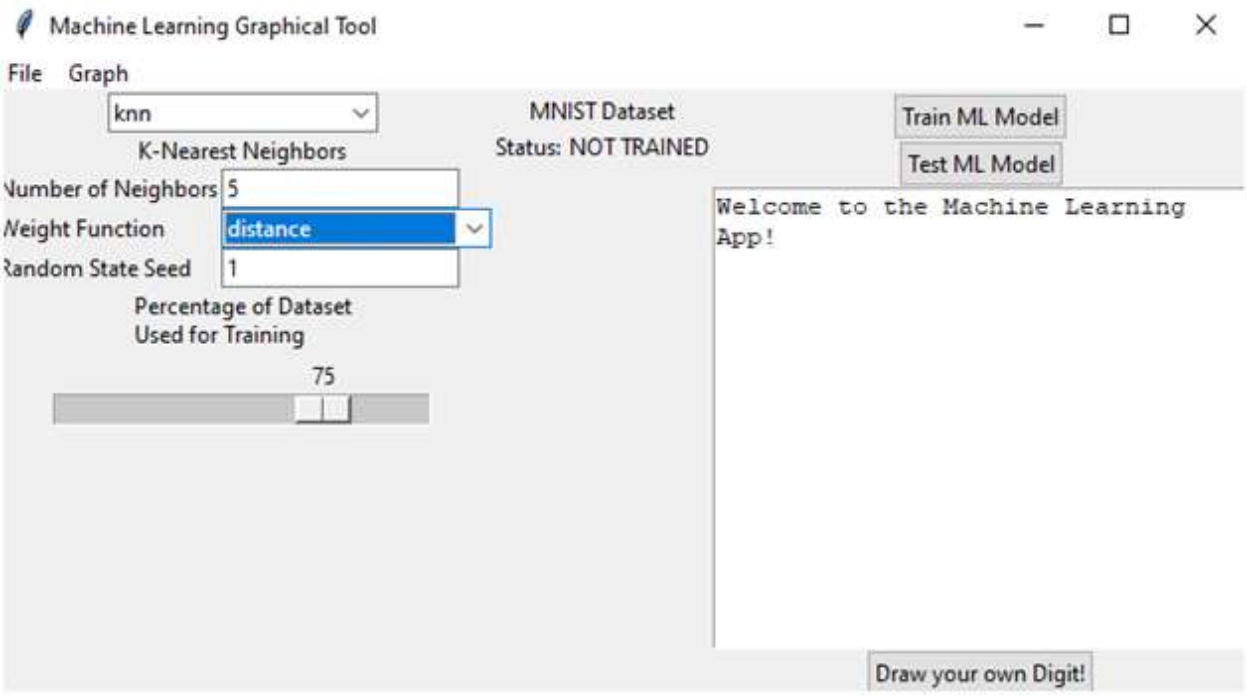
Test Cases

Test Case 1: Selecting a Learning Model - KNN			
Input	**Expected Output**	**Output**	**Result**
"K-Nearest Neighbor" in the Machine Learning Dropdown Box	Input boxes: Number of Neighbors (numerical text input) and Weight Function (dropdown box) displayed on the left side of the screen.	Input boxes: Number of Neighbors (numerical text input) and Weight Function (dropdown box) displayed on the left side of the screen.	Successfully selected the 'knn' from the KNN learning model.

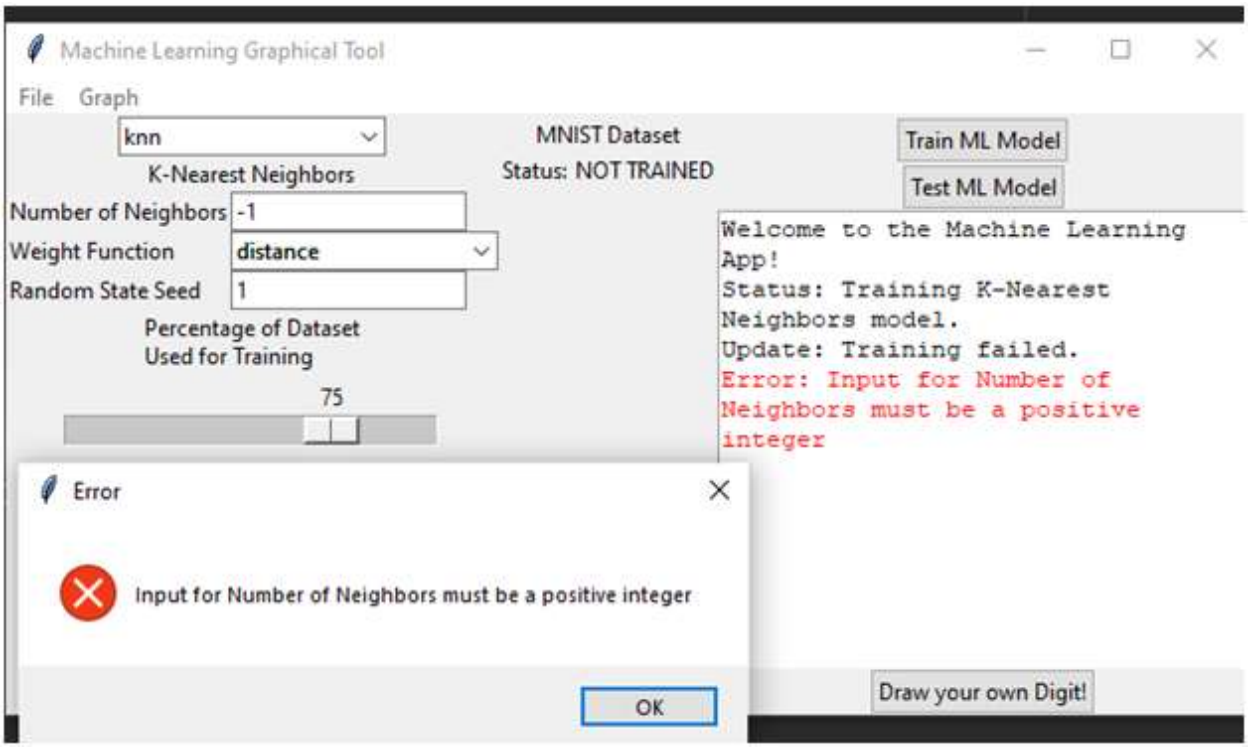
Test Case 2: Selecting the "Uniform" Weight Function for the KNN Learning Model			
Input	**Expected Output**	**Output**	**Result**
Select the "uniform" weight function for KNN Learning Model.	Successfully select the "uniform" weight function from the KNN Learning Model.	Successfully select the "uniform" weight function from the KNN Learning Model.	Pass



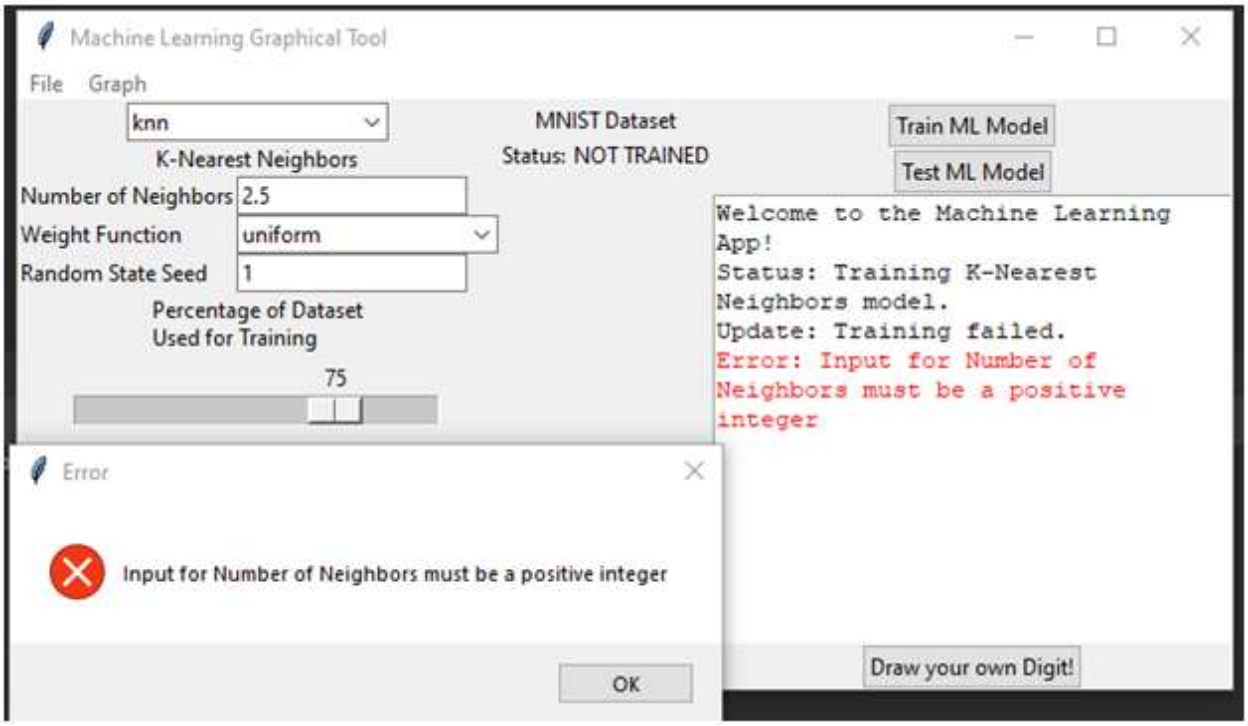
Test Case 3: Selecting the "Distance" Weight Function for the KNN Learning Model			
Input	**Expected Output**	**Output**	**Result**
Select the "distance" weight function for KNN Learning Model.	To be able to select the "distance" weight function from the KNN Learning Model.	Successfully selected the "distance" weight function from the KNN Learning Model.	Pass



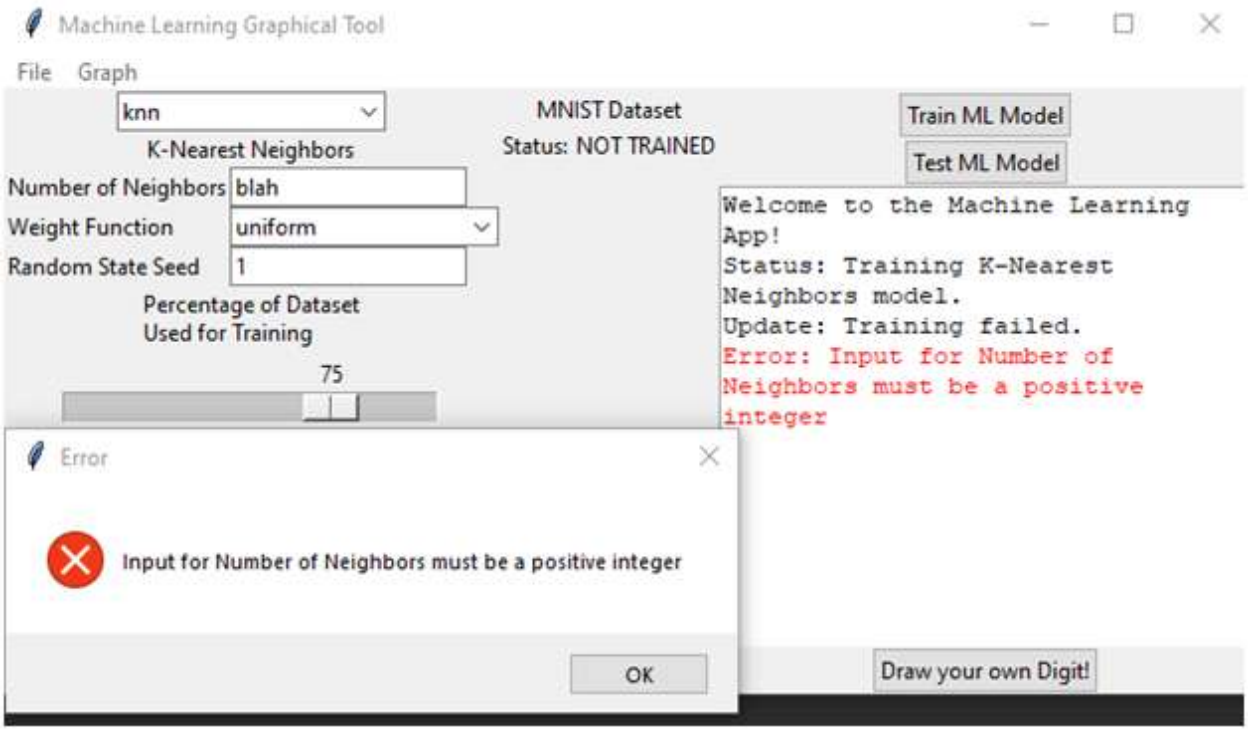
Test Case 4: Inputting "Number of Neighbors" parameter for the KNN Learning Model: Negative Number			
Input	**Expected Output**	**Output**	**Result**
Input the number "-1" in the "Number of Neighbors" text box.	Receive an error message informing the user that a positive integer is required for the program to work.	The error message "Input for Number of Neighbors must be a positive integer" is successfully displayed.	Pass



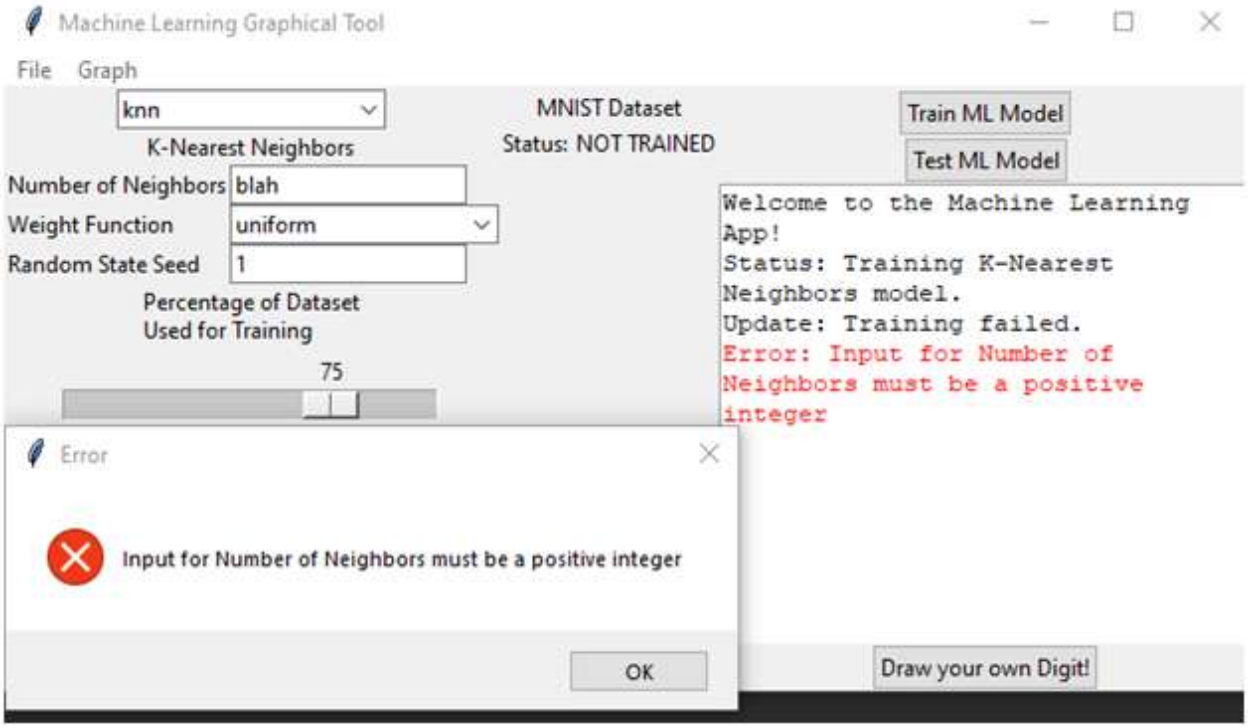
Test Case 5: Inputting "Number of Neighbors" parameter for the KNN Learning Model: Decimal			
Input	**Expected Output**	**Output**	**Result**
Input the number "2.5" in the "Number of Neighbors" text box.	Receive an error message informing the user that a positive integer is required for the program to work.	The error message "Input for Number of Neighbors must be a positive integer" is successfully displayed.	Pass



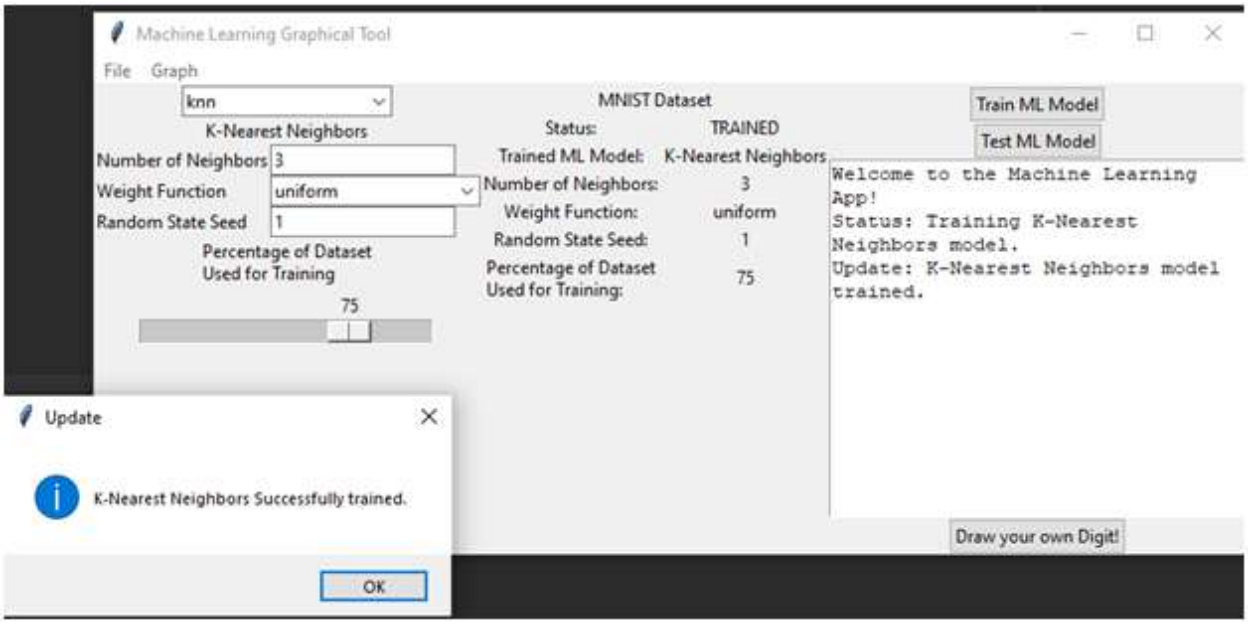
Test Case 6: Inputting "Number of Neighbors" parameter for the KNN Learning Model: Non-Numeric			
Input	**Expected Output**	**Output**	**Result**
Input the word "blah" in the "Number of Neighbors" text box.	Receive an error message informing the user that a positive integer is required for the program to work.	The error message "Input for Number of Neighbors must be a positive integer" is successfully displayed.	Pass



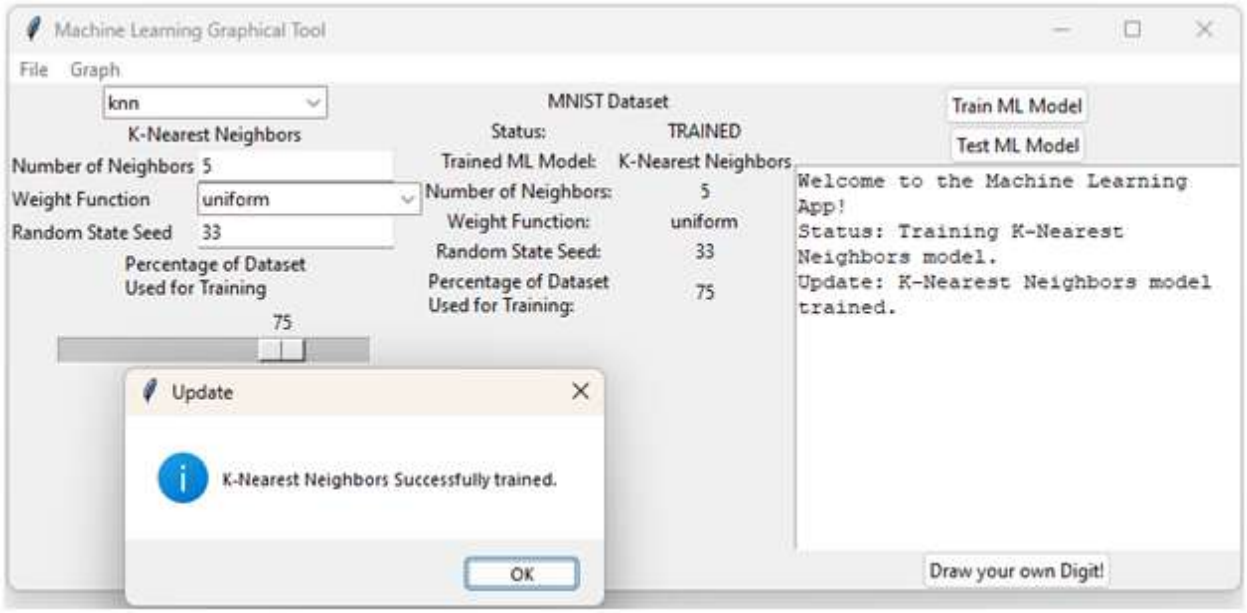
Test Case 7: Inputting "Number of Neighbors" parameter for the KNN Learning Model: Zero			
Input	**Expected Output**	**Output**	**Result**
Input the number "0" in the "Number of Neighbors" text box.	Receive an error message informing the user that a positive integer is required for the program to work.	Error message, "Input for Number of Neighbors must be a positive integer" is displayed.	Pass



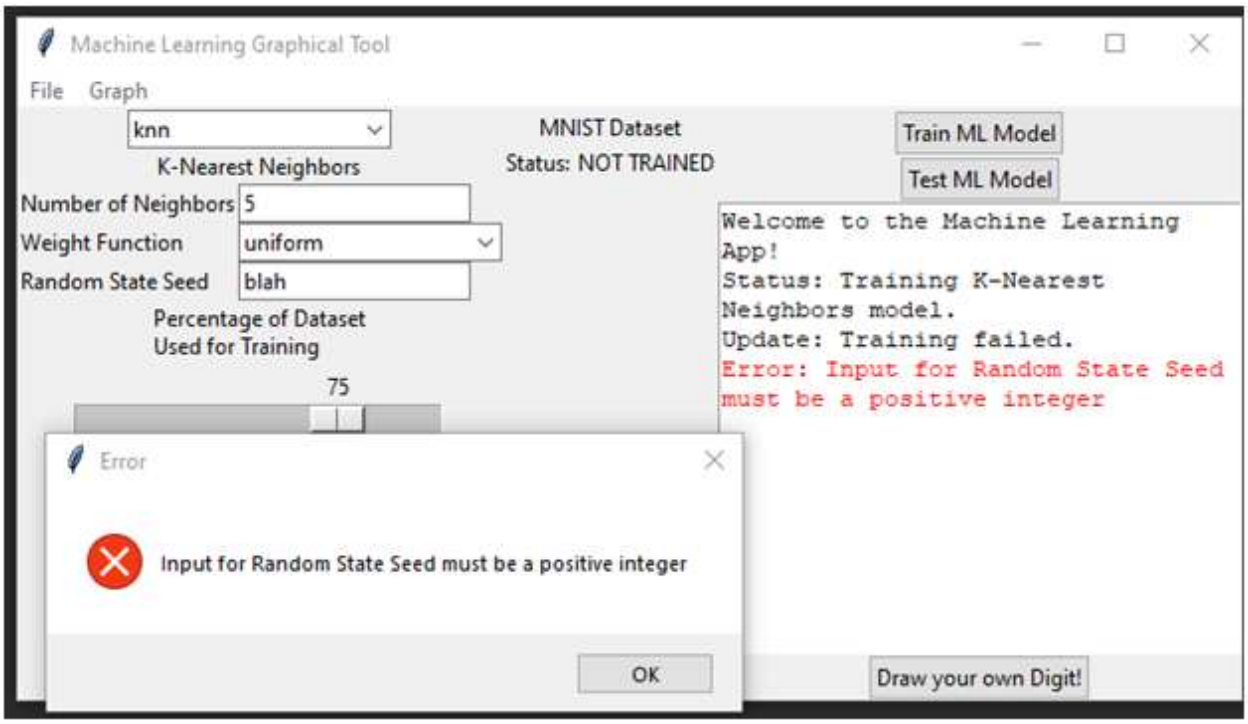
Test Case 8: Inputting "Number of Neighbors" parameter for the KNN Learning Model: Positive Integer			
Input	**Expected Output**	**Output**	**Result**
Enter the number "3" in the "Number of Neighbors" text box.	Successfully set the "Number of Neighbors" parameter to "3" for the KNN learning model.	The "Number of Neighbors" parameter is successfully set to "3" for the KNN learning model accompanied by the message "K-Nearest Neighbors Successfully trained."	Pass



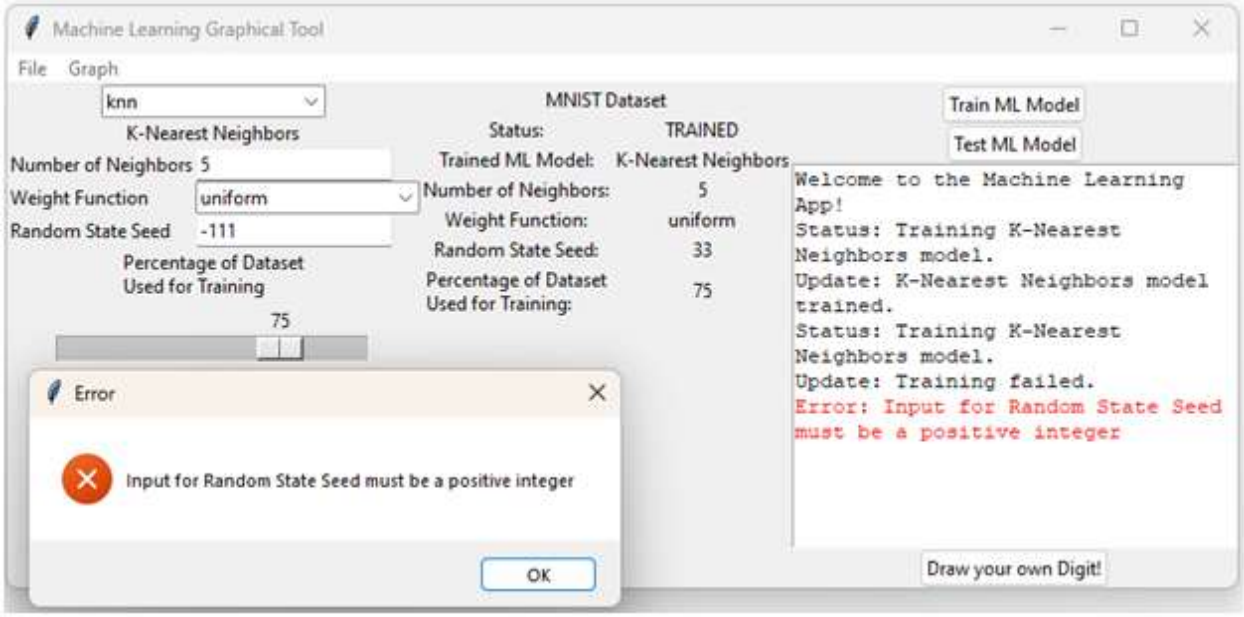
Test Case 9: Test Random State Seed: Positive Integer			
Input	**Expected Output**	**Output**	**Result**
1.) Input 33 in the Random State Seed text box. 2.) Press "Train ML Model"	KNN model will train with "Random State Seed" showing a value of 33 in the center panel. .	As Expected	Pass



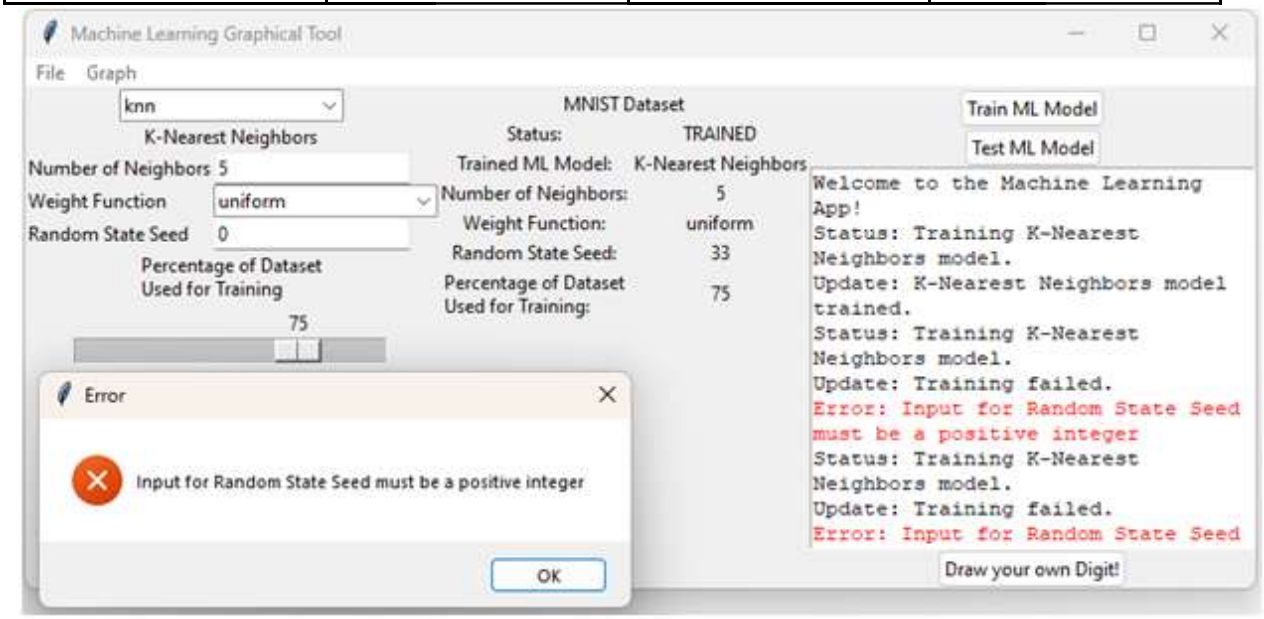
Test Case 10: Random State Seed: Non-Numeric Value			
Input	**Expected Output**	**Output**	**Result**
Input the word “blah” in the “Random State Seed” text box.	Error message indicating that a positive number less than 100 is required.	Error message stating, “Input for Random State Seed must be a positive integer”.	Pass



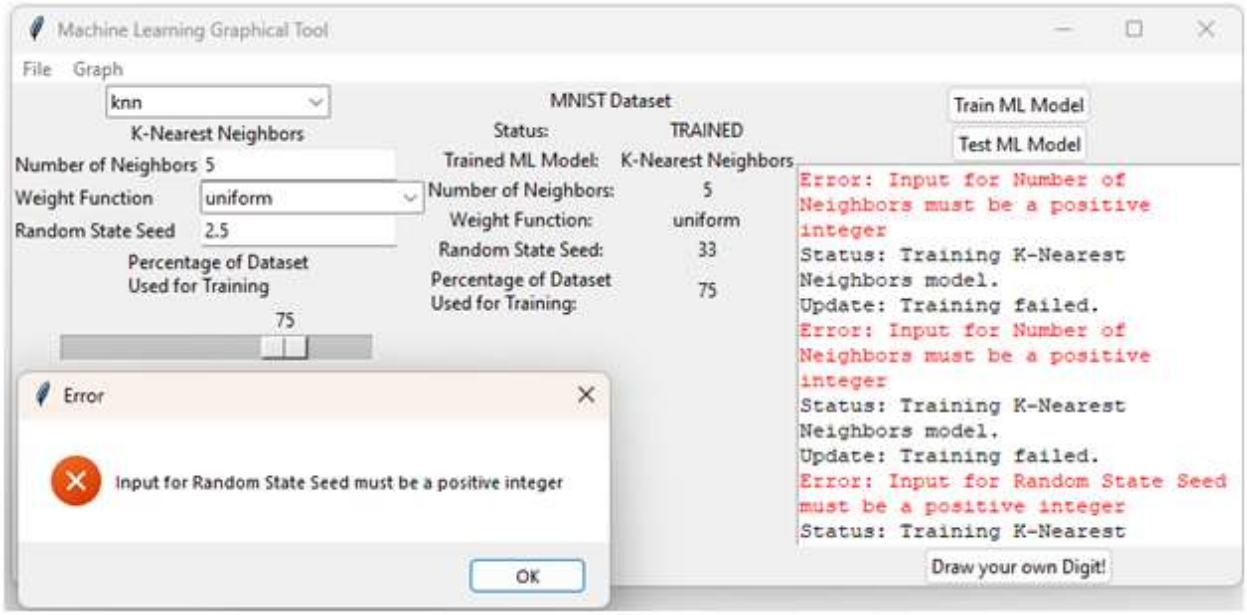
Test Case 11: Random State Seed: Negative Value			
Input	**Expected Output**	**Output**	**Result**
1. Input "-111" (representing 111%) in the "Random State Seed" input box. 2. Press "Train ML Model"	Error message indicating that a positive integer is required	As Expected	Pass



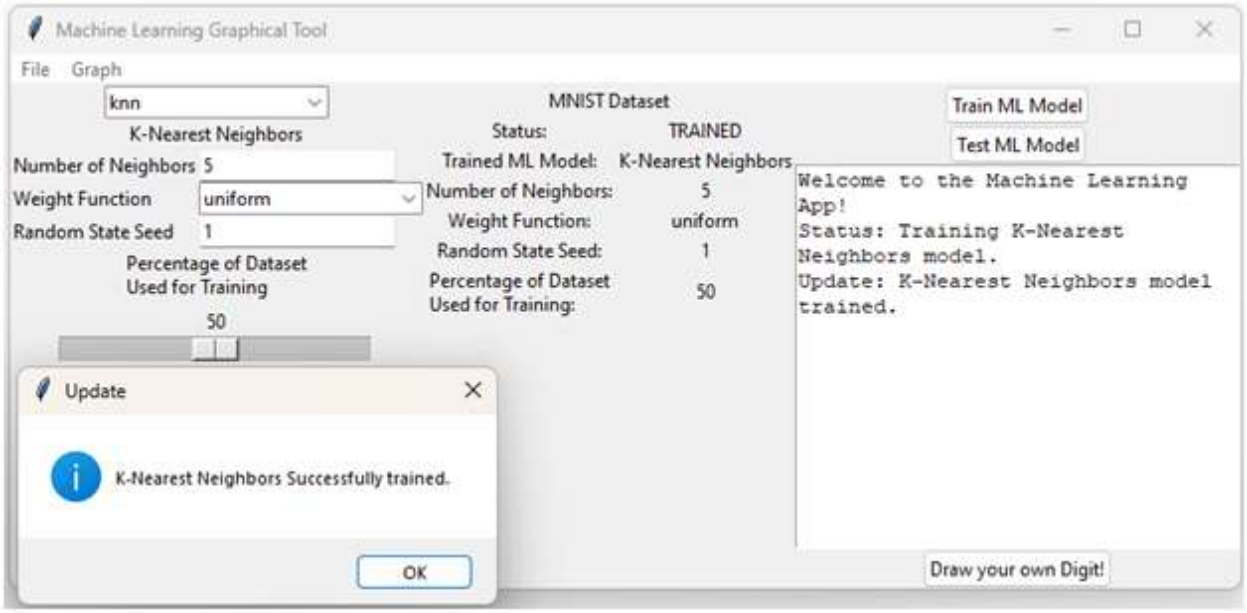
Test Case 12: Random State Seed: 0			
Input	**Expected Output**	**Output**	**Result**
1. Input “0” in the “Random State Seed” input box. 2. Press “Train ML Model”	Error message indicating that a positive integer is required	As Expected	Pass



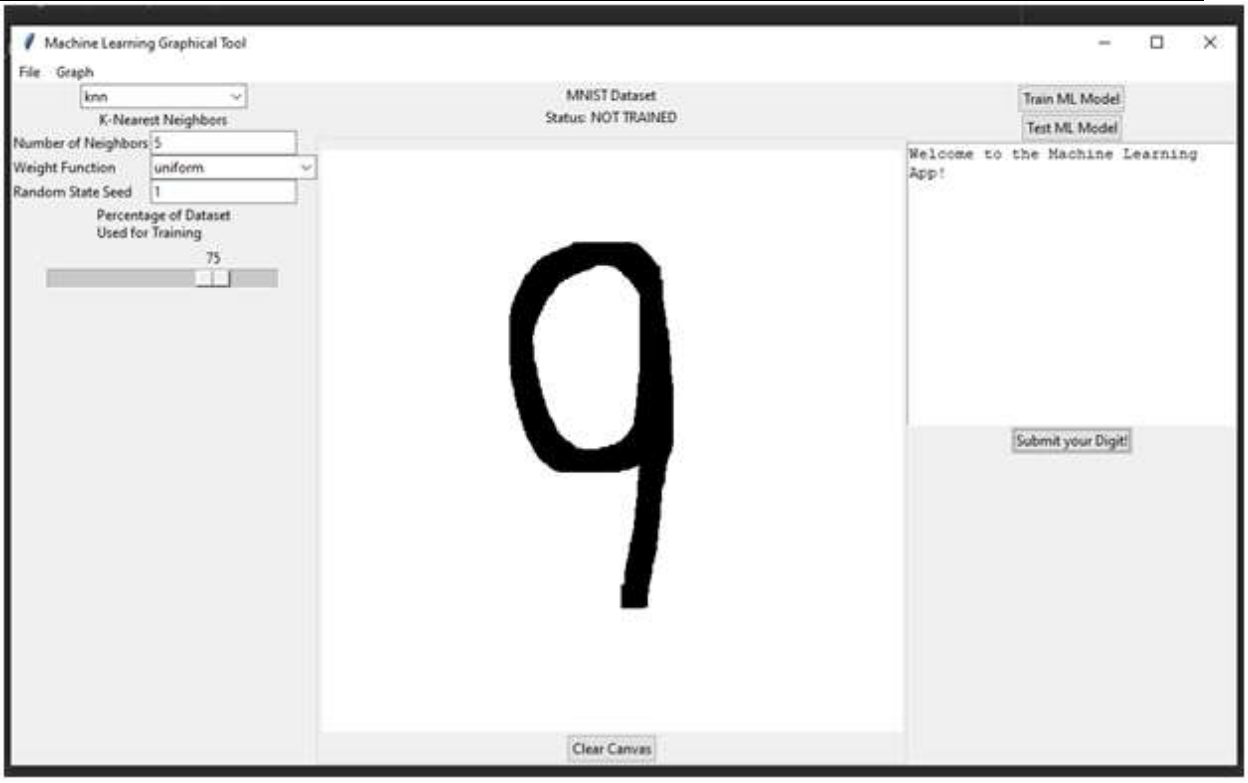
Test Case 13: Random State Seed: Decimal			
Input	**Expected Output**	**Output**	**Result**
1. Input “2.5” in the “Random State Seed” input box. 2. Press “Train ML Model”	Error message indicating that a positive integer is required	As Expected	Pass



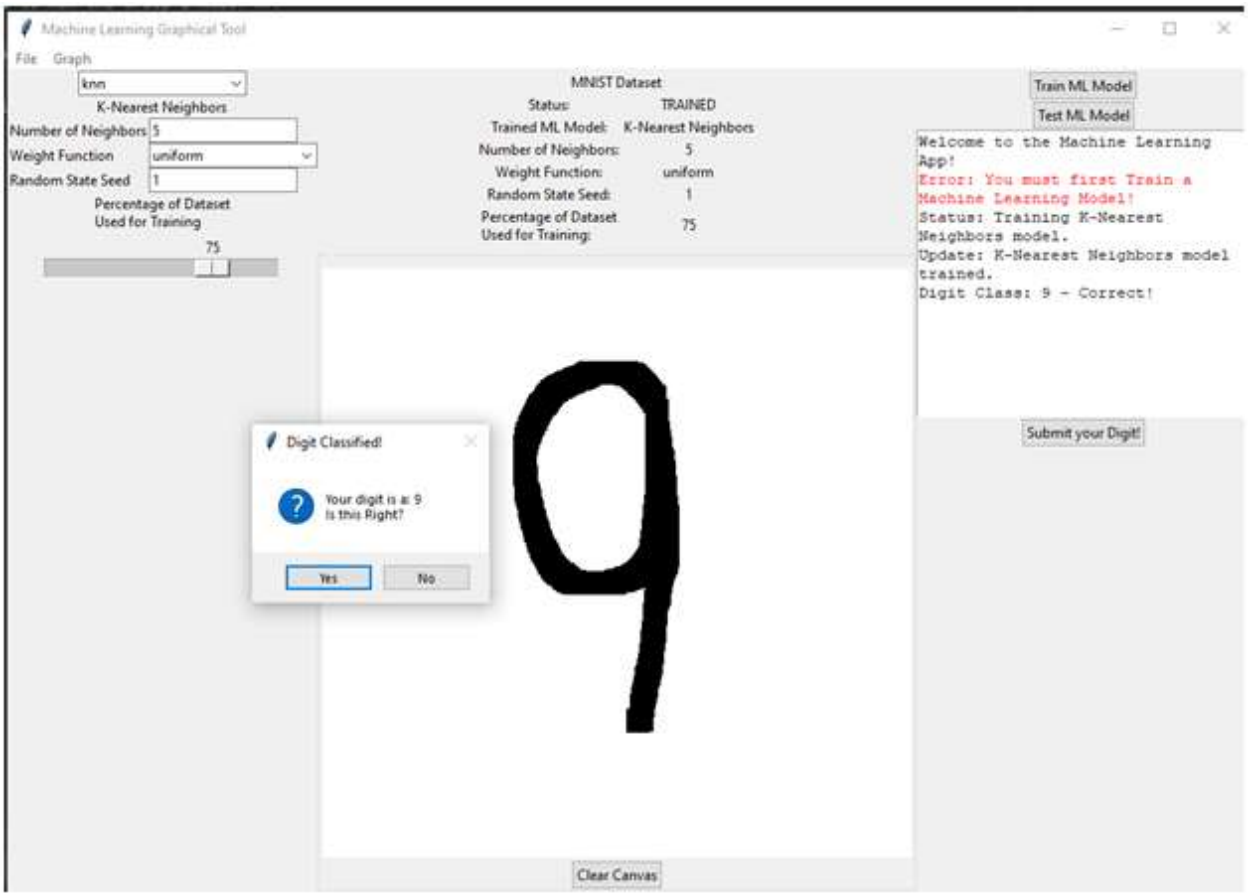
Test Case 14: Partition the Dataset: 50			
Input	**Expected Output**	**Output**	**Result**
Input “50” (representing partition of the dataset into equal sized training and testing subsets) in the “Percentage of Dataset Used for Training” Sliding Scale	The dataset is partitioned with message “K-Nearest Neighbors Successfully trained”. The center panel will show “50” to the right of “Percentage of Dataset Used for Training”	Dataset partitioned with the message “K-Nearest Neighbors Successfully trained”. Center panel display as expected.	Pass



Test Case 15: Hand Drawn Digit Input			
Input	**Expected Output**	**Output**	**Result**
Click and drag a digit on the input canvas.	Black pixels of a digit are seen on the input canvas.	Black pixels in the shape of the number “9” were successfully drawn/ seen on the input canvas.	Pass



Test Case 16: User-Drawn Digit Submission			
Input	**Expected Output**	**Output**	**Result**
Select the "Submit your Digit" button.	Display message box called "Digit Classified" with the messages "Your digit is a:" and the digit believed to have been drawn. "Is that Right?" with the options "Yes" and "No". On the right side, the digit guessed ("9") with the word "Correct!"	Successfully displayed the correct digit drawn "9", "Digit Classified" message box and confirmation message.	Pass

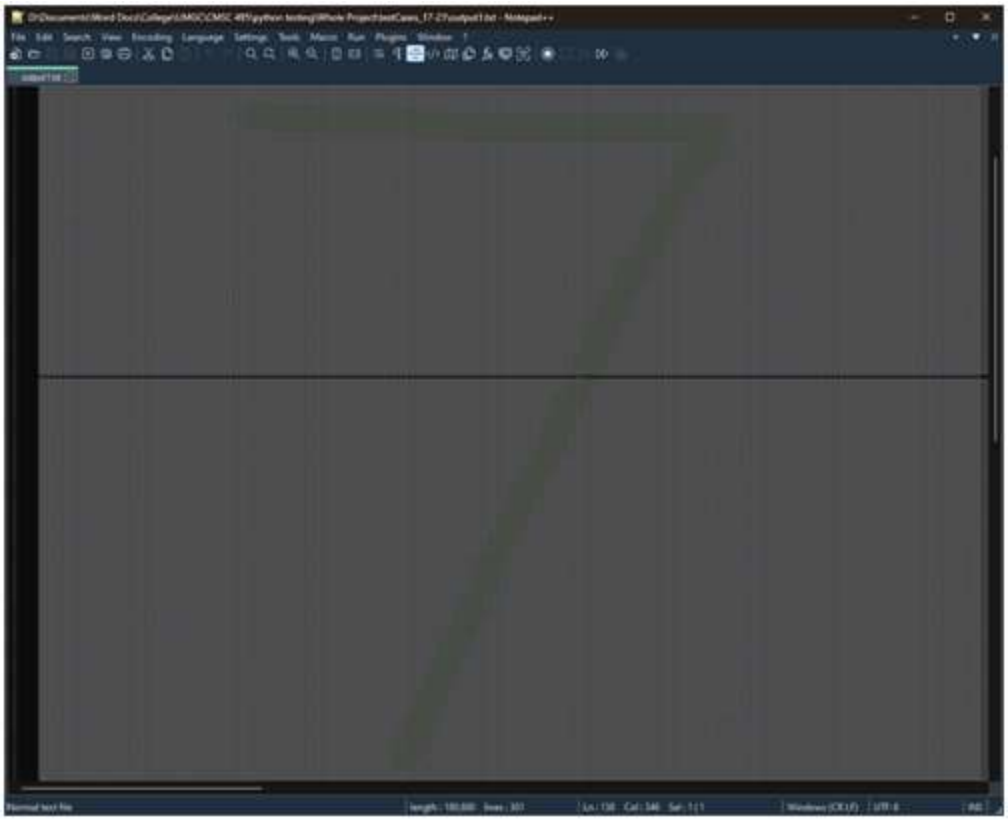


NOTE: For Test Cases 17-21, please refer to the files in the "test" folder of the MNIST-App directory.

These test cases are run by executing the "ImgMatTest.py" file in that folder.

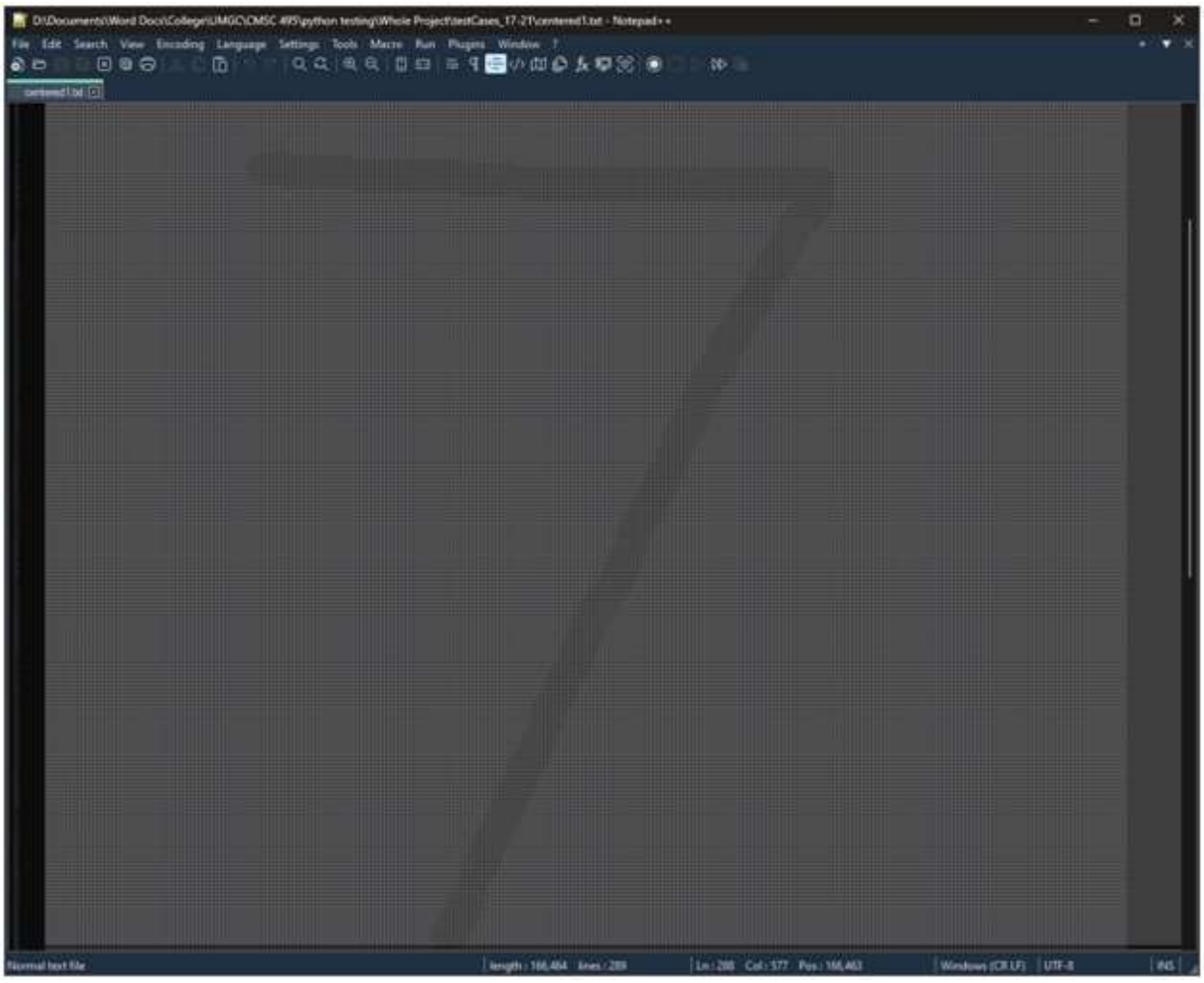
Test Case 17: Digit Image to Integer Matrix Conversion			
Input	**Expected Output**	**Actual Output**	**Result**
a 300x300 black and white bitmap file representing the digit "7" with a horizontal line through its approximate center. (See attached file, "digit7.bmp")	an integer matrix, with the same number of rows and columns as in the input bitmap (300 rows, 300 column) will be returned, where for every value "x" and "y", matrix[x][y] will equal 0 if it corresponds to a white pixel on the input bitmap, or 1 if it corresponds to a black pixel on the input bitmap. (See attached file, "output1.txt")	As Expected. Note the screenshot from Notepad++ below. It is from the file "output1.txt" in "test" folder.	Pass

Output: A 300 line, 600 column text file made up of "1"s and "0"s separated by whitespace, with the "1"s in the shape of the 7 drawn in "digit7.bmp"



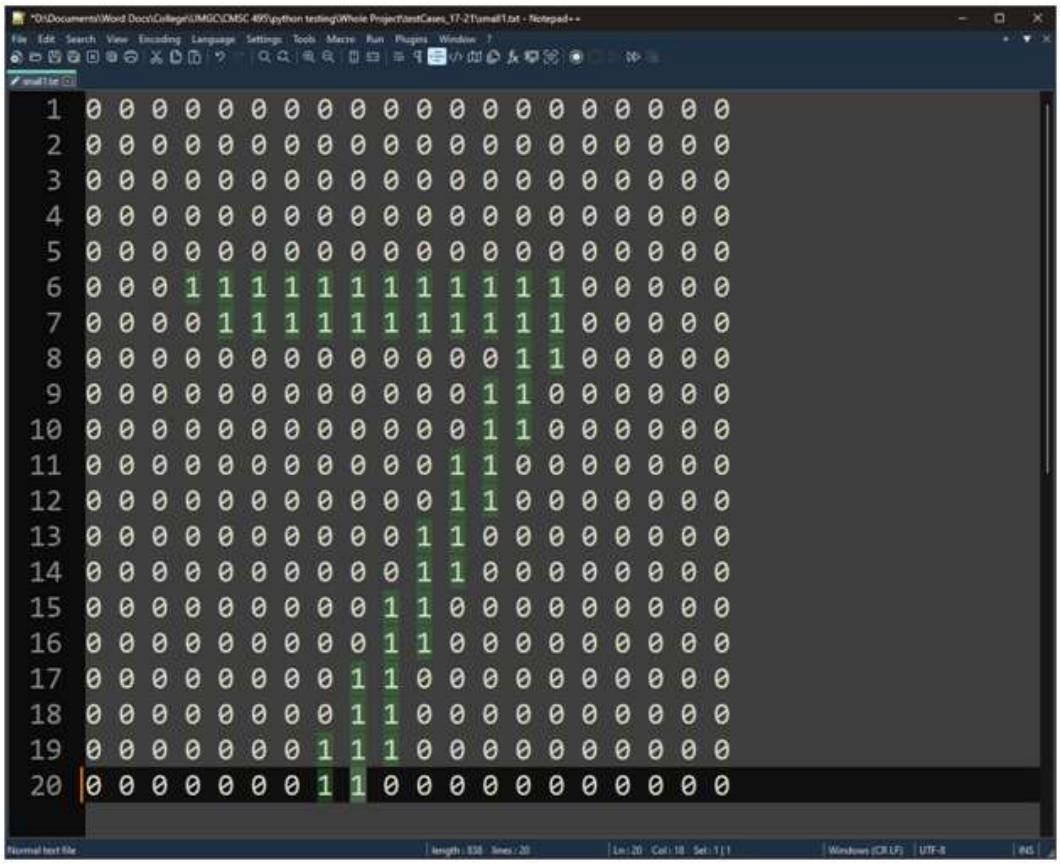
Test Case 18: Digit Centering			
Input	**Expected Output**	**Actual Output**	**Result**
a 300x300 black and white bitmap file representing the digit “7” with a horizontal line through its approximate center. (See attached file, “digit7.bmp”)	The binary integer matrix from Test Case 17 will be centered about the center of mass of the matrix values. This will be displayed as a text file made of “1”s and “0's separated by white space.	As Expected. Note the screenshot from Notepad++ below. It is from the file “centered1.txt” in “test” folder.	Pass

Output: A 288 line, 500 column text file made up of “1”s and “0”s separated by whitespace, with the “1”s in the shape of the 7 drawn in “digit7.bmp” The seven is centered about its center of mass.



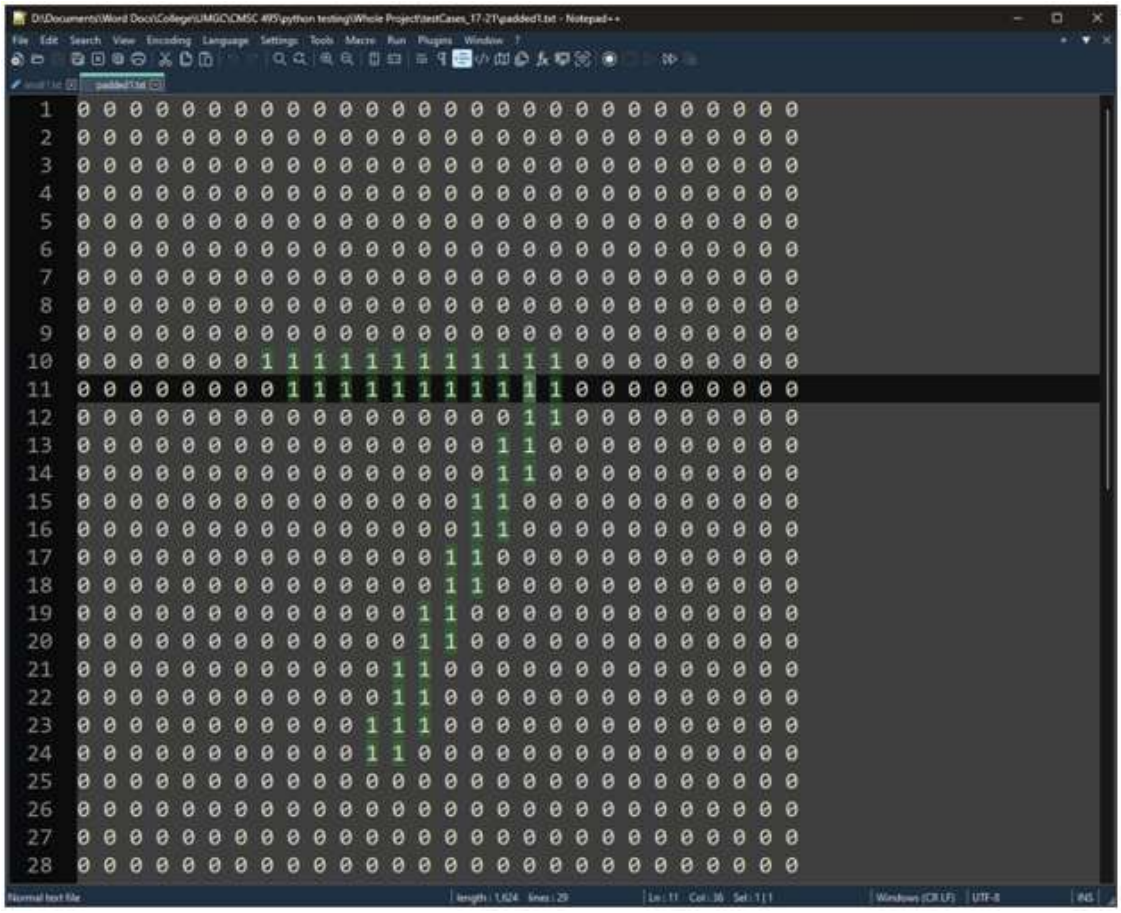
Test Case 19: Digit Resizing			
Input	**Expected Output**	**Actual Output**	**Result**
a 300x300 black and white bitmap file representing the digit “7” with a horizontal line through its approximate center. (See attached file, “digit7.bmp”)	The centered binary integer matrix from Test Case 18 will be reduced in resolution to 20x20 elements. This will be displayed as a text file made of “1”s and “0”s separated by white space, of 20 rows and 40 columns.	As Expected. Note the screenshot from Notepad++ below. It is from the file “small1.txt” in “test” folder.	Pass

Output: A 20 line, 40 column text file made up of “1”s and “0”s separated by whitespace, with the “1”s in the shape of the 7 drawn in “digit7.bmp” The seven is centered about its center of mass. It is a reduced resolution form of the matrix from Test Case 18.



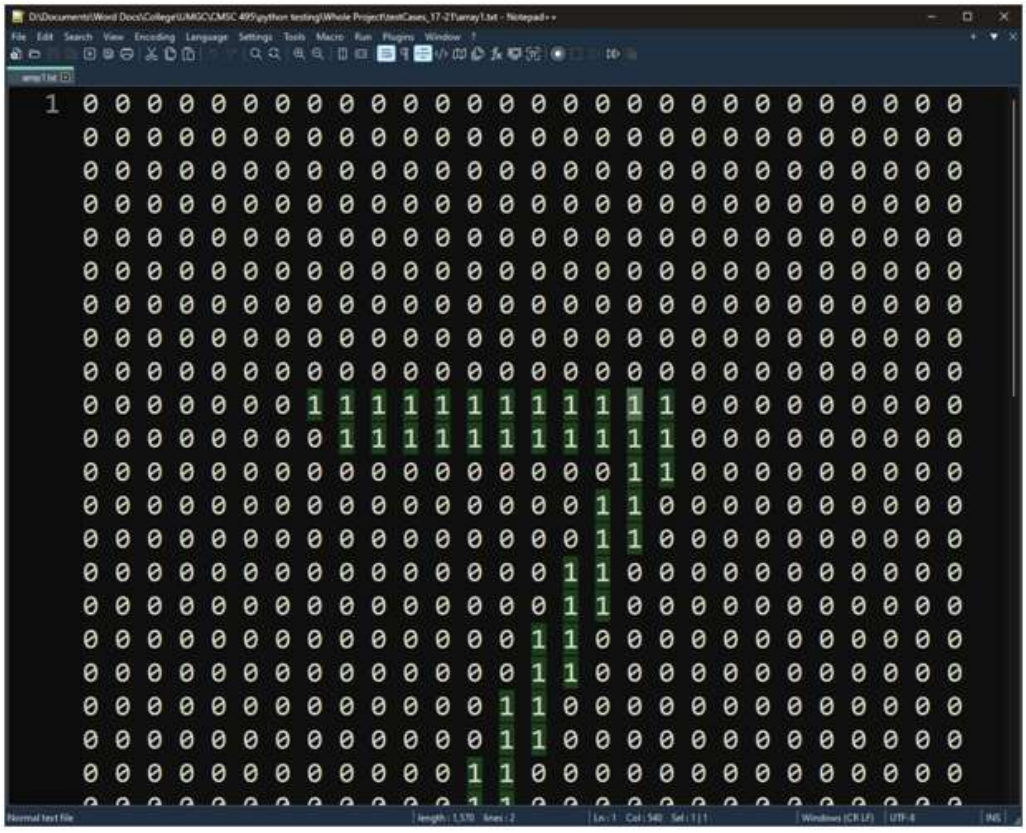
Test Case 20: Digit Resizing			
Input	**Expected Output**	**Actual Output**	**Result**
a 300x300 black and white bitmap file representing the digit “7” with a horizontal line through its approximate center. (See attached file, “digit7.bmp”)	The reduced, centered binary integer matrix from Test Case 19 will be “padded” with 4 rows and 4 columns of 0s about each side, to make it compatible with the MNIST dataset elements. This will be displayed as a text file made of “1”s and “0”s separated by white space, of 28 rows and 56 columns.	As Expected. Note the screenshot from Notepad++ below. It is from the file “padded1.txt” in “test” folder.	Pass

Output: A 28 line, 56 column text file made up of “1”s and “0”s separated by whitespace, with the “1”s in the shape of the 7 drawn in “digit7.bmp” The seven is centered about its center of mass. It is similar to the matrix from Test Case 19, but “padded” with four rows and columns of 0s about each side.



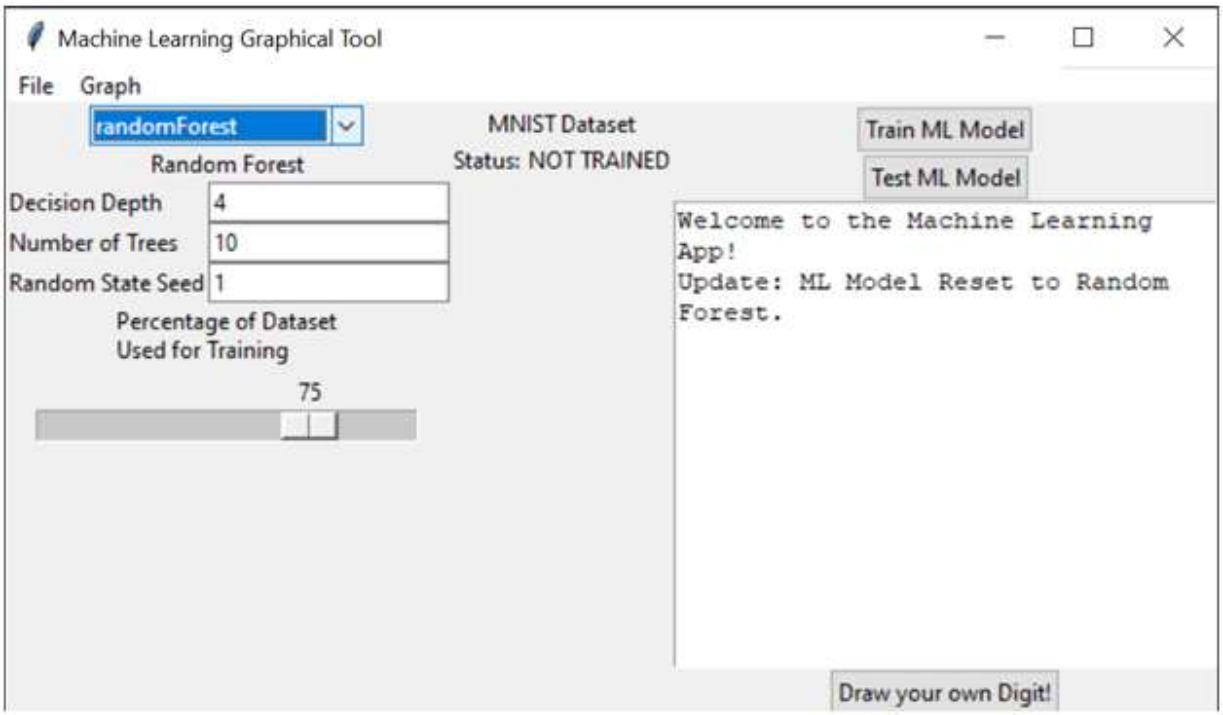
Test Case 21: Padded Digit Conversion to Array			
Input	**Expected Output**	**Actual Output**	**Result**
a 300x300 black and white bitmap file representing the digit “7” with a horizontal line through its approximate center. (See attached file, “digit7.bmp”)	The padded, reduced, centered binary integer matrix from Test Case 20 will be converted, in Row-Major Order, into a 1-Dimensional Array to make it compatible with the MNIST dataset elements. This will be displayed as a text file made of “1”s and “0”s separated by white space, of 1 row and 1560 columns.	As Expected. Note the screenshot from Notepad++ below. It is from the file “array1.txt” in “test” folder.	Pass

Output: A 1 line, 1560 column text file made up of “1”s and “0”s separated by whitespace, with the “1”s in the shape of the 7 drawn in “digit7.bmp”. Note the single “1” in the top-left indicates this is a text file of one row, i.e., it is a one-dimensional array based on a row-major transformation of the matrix from test case 20.



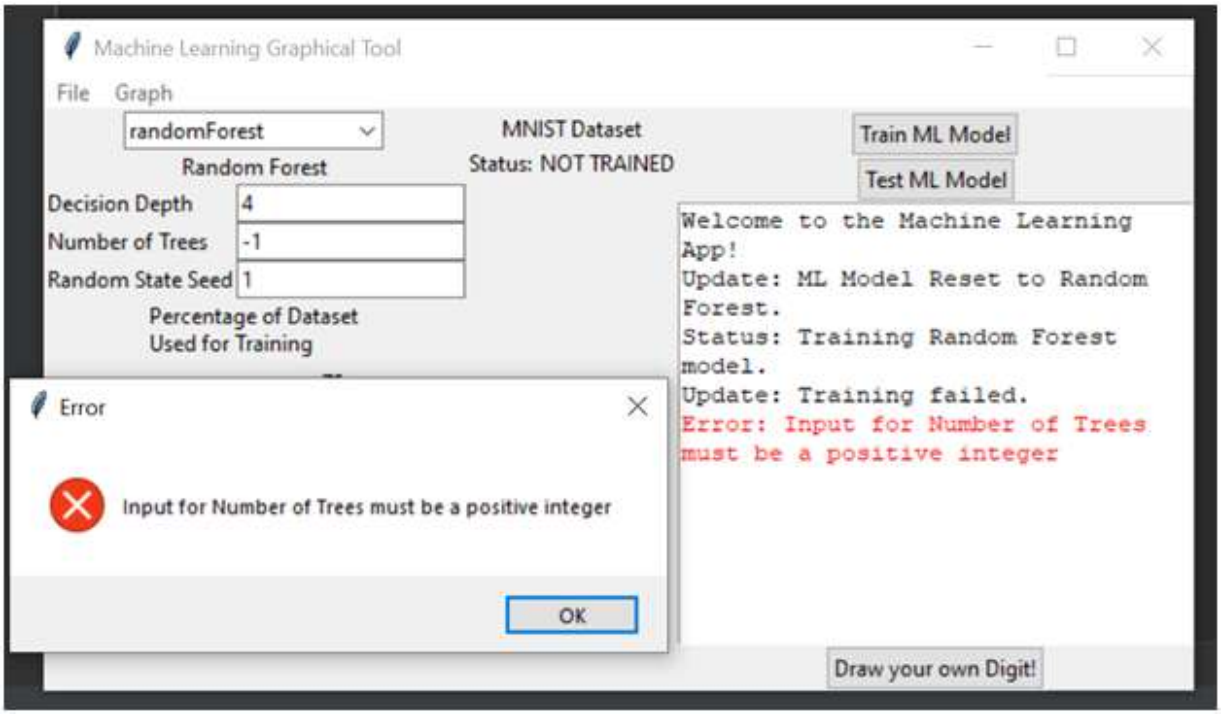
Test Case 22 – Selecting a Learning Model – Random Forest

Input	Expected Output	Output	Result
From the drop-down menu, Select "randomForest"	The program loads the Random Forest Classifier component and its unique parameters, Number of Estimators (trees), and Max Decision Depth.	The app loaded the Random Forest Classifier and provided a status update in the log box: "Update: ML Model Reset to Random Forest." The user-adjustable parameters now also reflect the Random Forest Model.	Pass



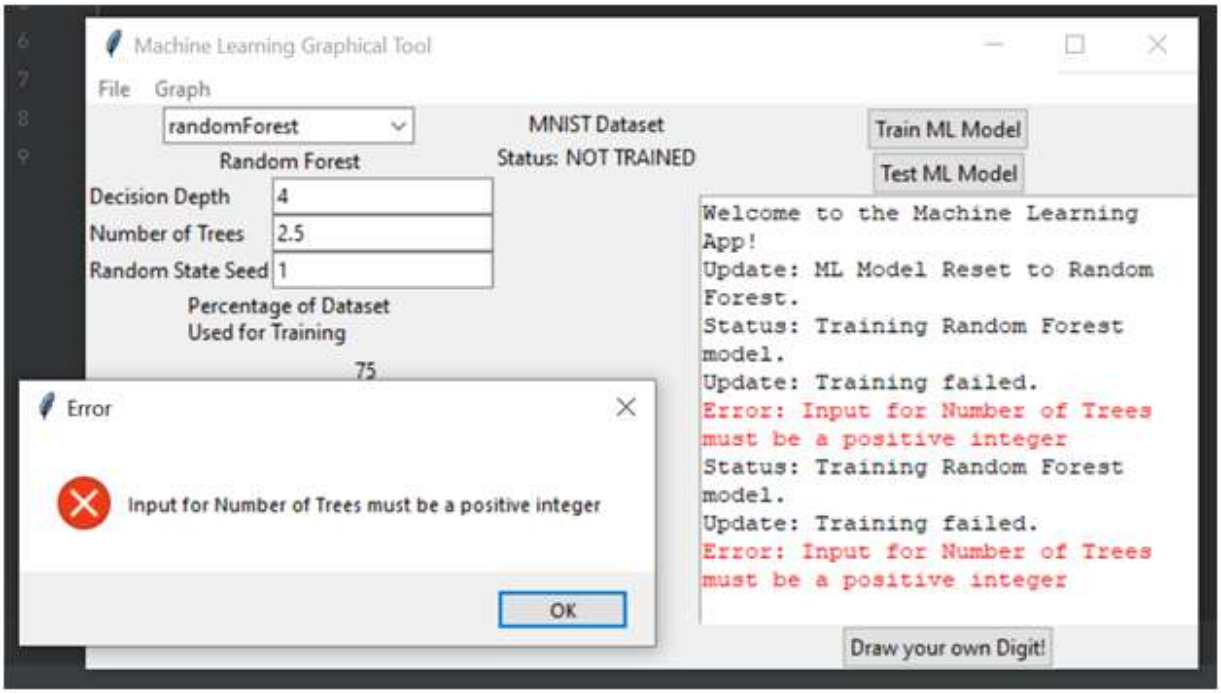
Test Case 23 – Inputting the "Number of Trees" parameter for the Random Forest Learning Model: Negative Number

Input	Expected Output	Output	Result
Input "-1" into the "Number of Trees" text box.	Error message, noting that a positive integer value is required for number of trees.	Error: Input for Number of Trees must be a positive integer	Pass



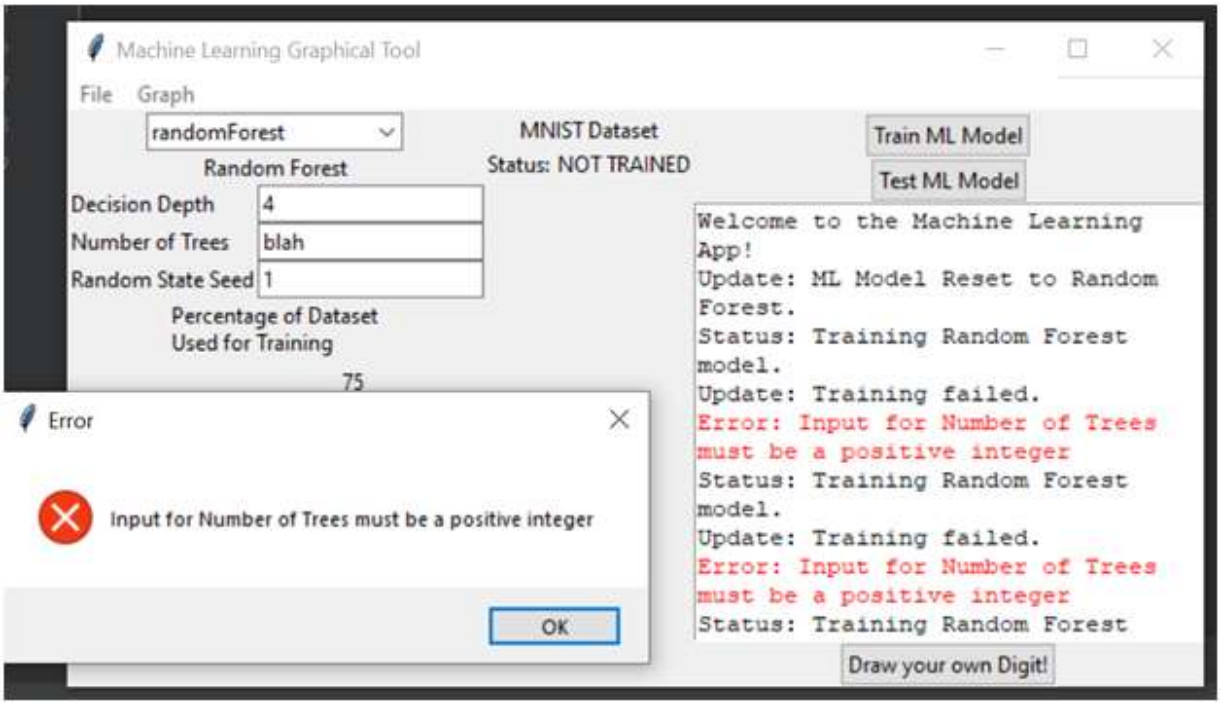
Test Case 24 – Inputting "Number of Trees" parameter for the Random Forest Learning Model: Decimal

Input	Expected Output	Output	Result
Input "2.5" in the "Number of trees" text box	Error message, noting that a positive integer value is required for number of trees.	Error: Input for Number of Trees must be a positive integer	Pass



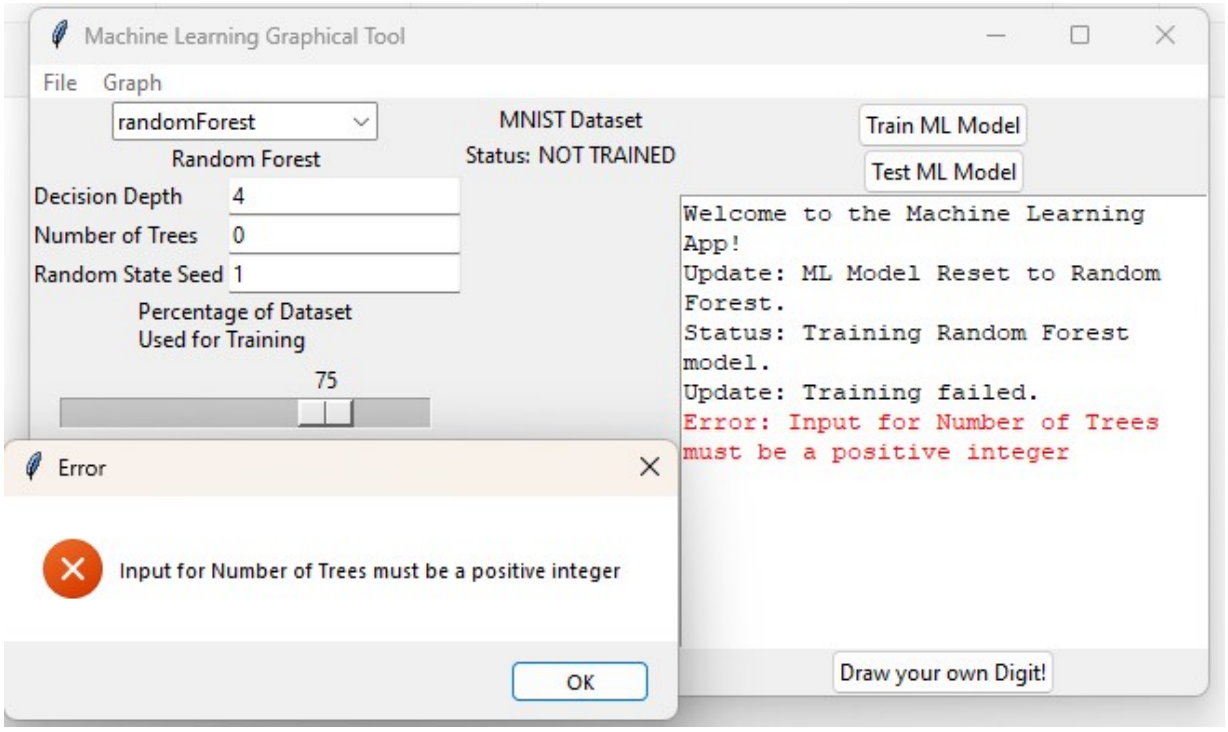
Test Case 25: Inputting "Number of Trees" parameter for the Random Forest Learning Model: Non-Numeric

Input	Expected Output	Output	Result
Input "blah" in the "Number of trees" text box	Error message, noting that a positive integer value is required for number of trees.	Error: Input for Number of Trees must be a positive integer	Pass



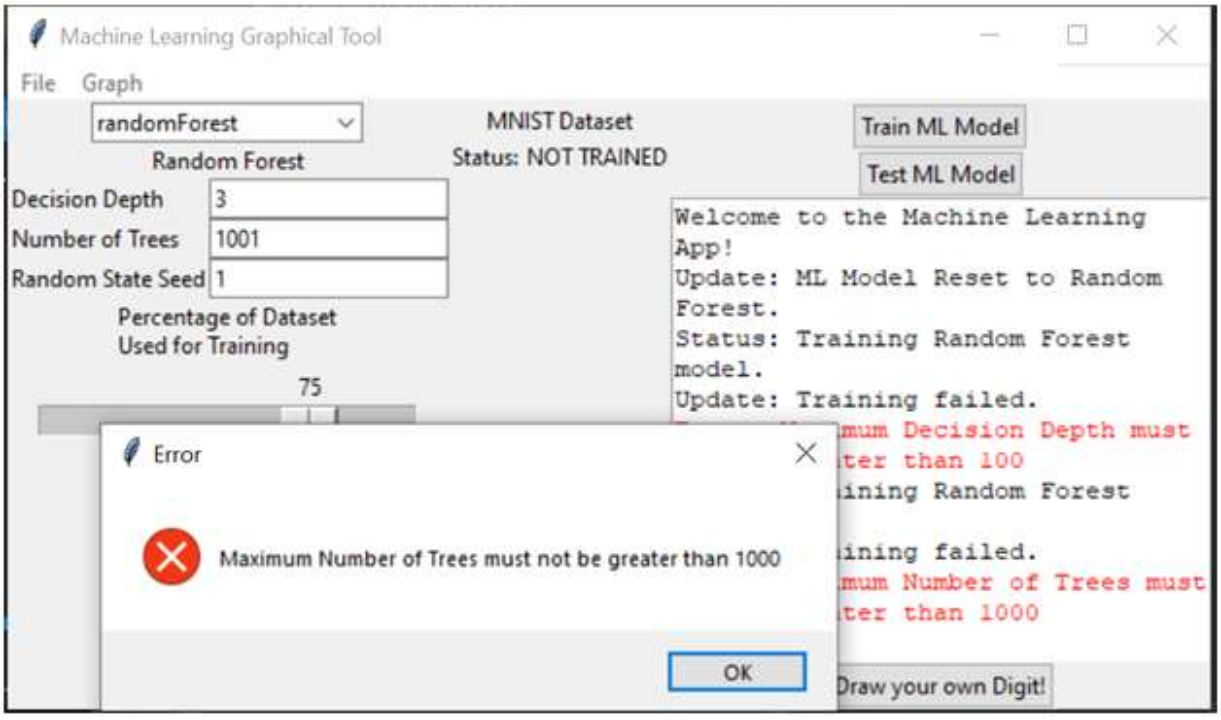
Test Case 26: Inputting "Number of Trees" parameter for the Random Forest Learning Model: Zero

Input	Expected Output	Output	Result
Input "0" in the "Number of trees" text box	Error message, noting that the number must be larger than 0 for the number of trees.	Error: Input for Number of Trees must be a positive integer	Pass



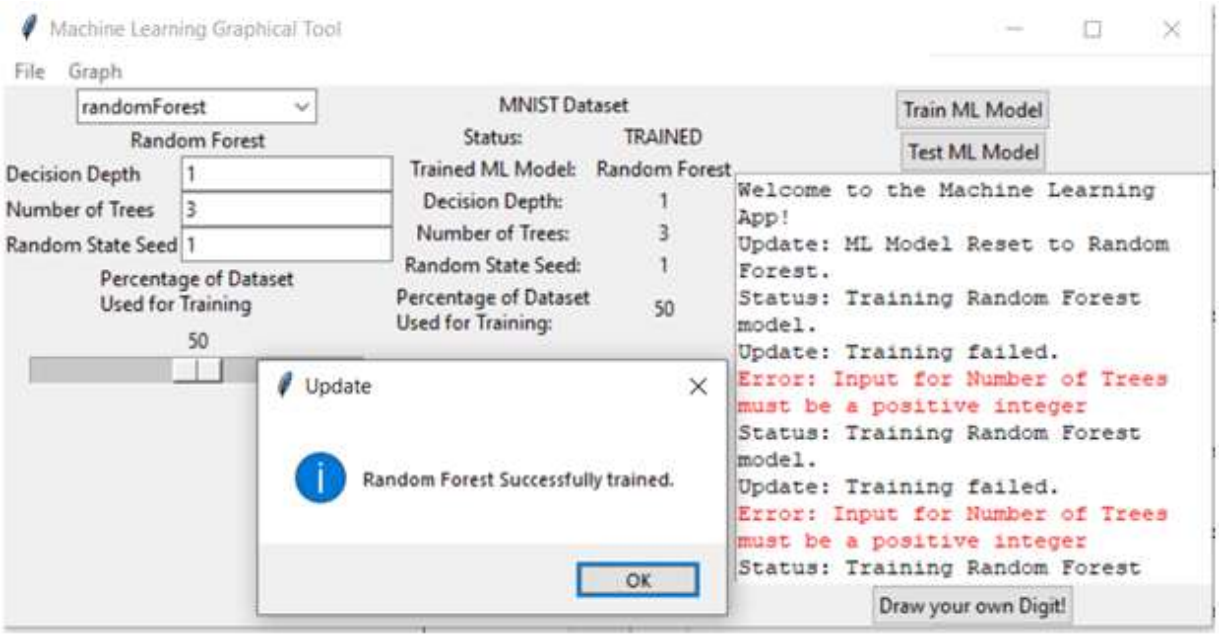
Test Case 27: Inputting "Number of Trees" parameter for the Random Forest Learning Model: > 1000

Input	Expected Output	Output	Result
Input "1001" in the "Number of trees" text box	Error message, noting that a positive integer value less than or equal to 1000 is required for the number of trees.	Error: Input for Number of Trees must be a positive Integer	Pass



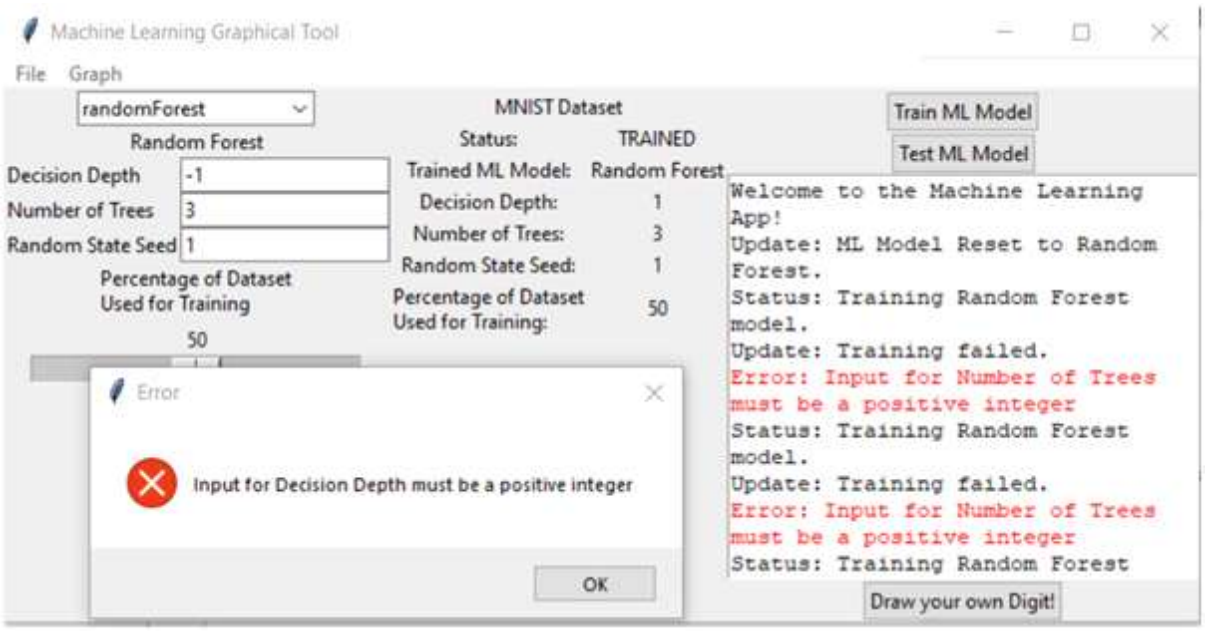
Test Case 28: Inputting "Number of Trees" parameter for the Random Forest Learning Model: Positive Integer

Input	Expected Output	Output	Result
Input "3" in the "Number of trees" text box	The "Number of Estimators" parameter will be set to "3" for the Random Forest learning model.	Random Forest Successfully trained. (with 3 trees)	Pass, but wrong error message



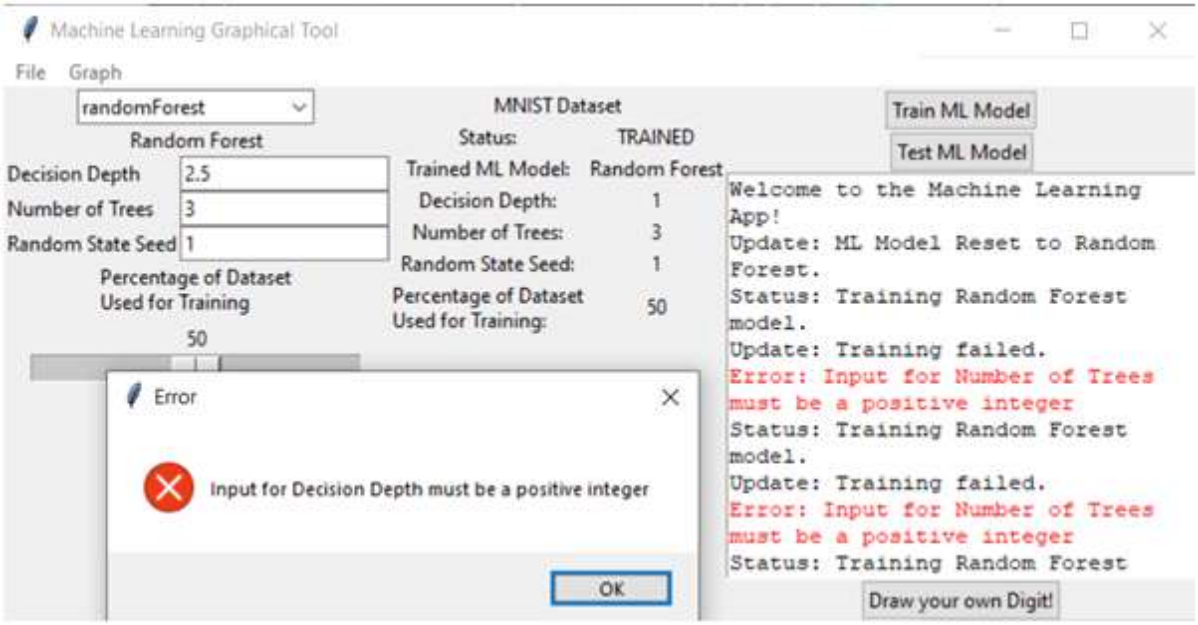
Test Case 29: Inputting "Max Decision Depth" parameter for the Random Forest Learning Model: Negative Number

Input	Expected Output	Output	Result
Input "-1" in the "Max Decision Depth" text box	Error message, noting that a positive integer value is required for max decision depth.	Input for Decision Depth must be a positive integer	Pass



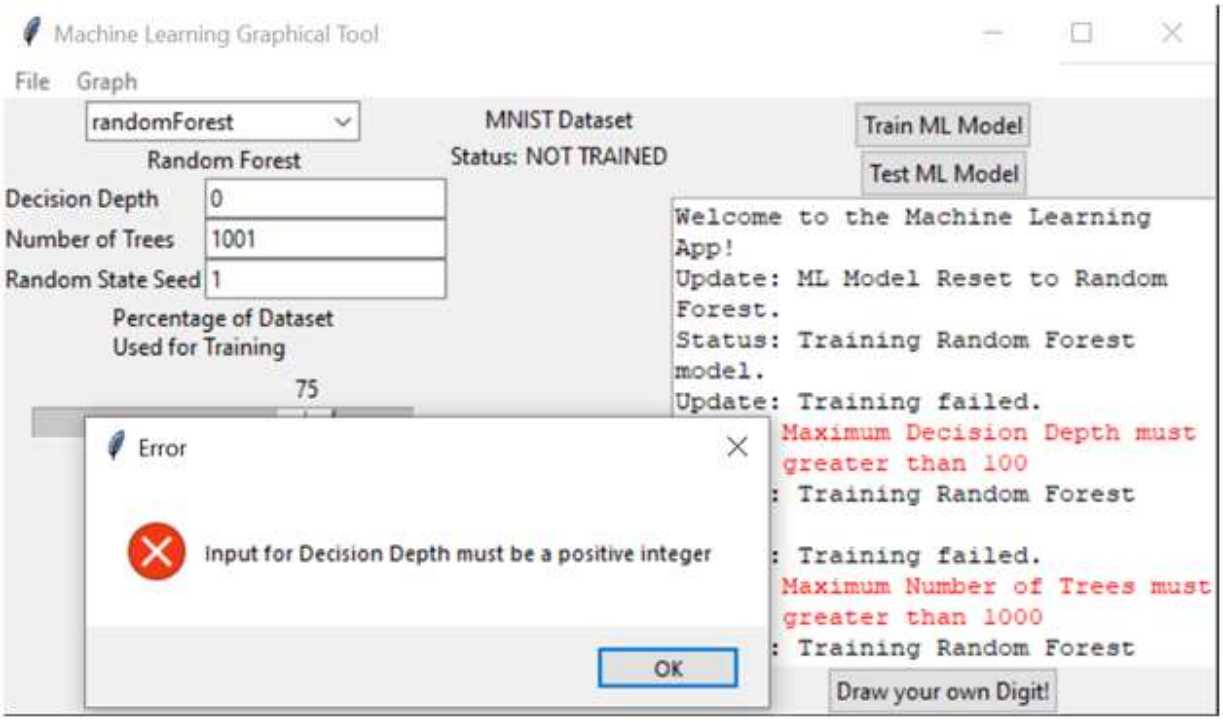
Test Case 30: Inputting "Max Decision Depth" parameter for the Random Forest Learning Model: Decimal

Input	Expected Output	Output	Result
Input "2.5" in the "Max Decision Depth" text box	Error message, noting that a positive integer value is required for max decision depth.	Input for Decision Depth must be a positive integer	Pass



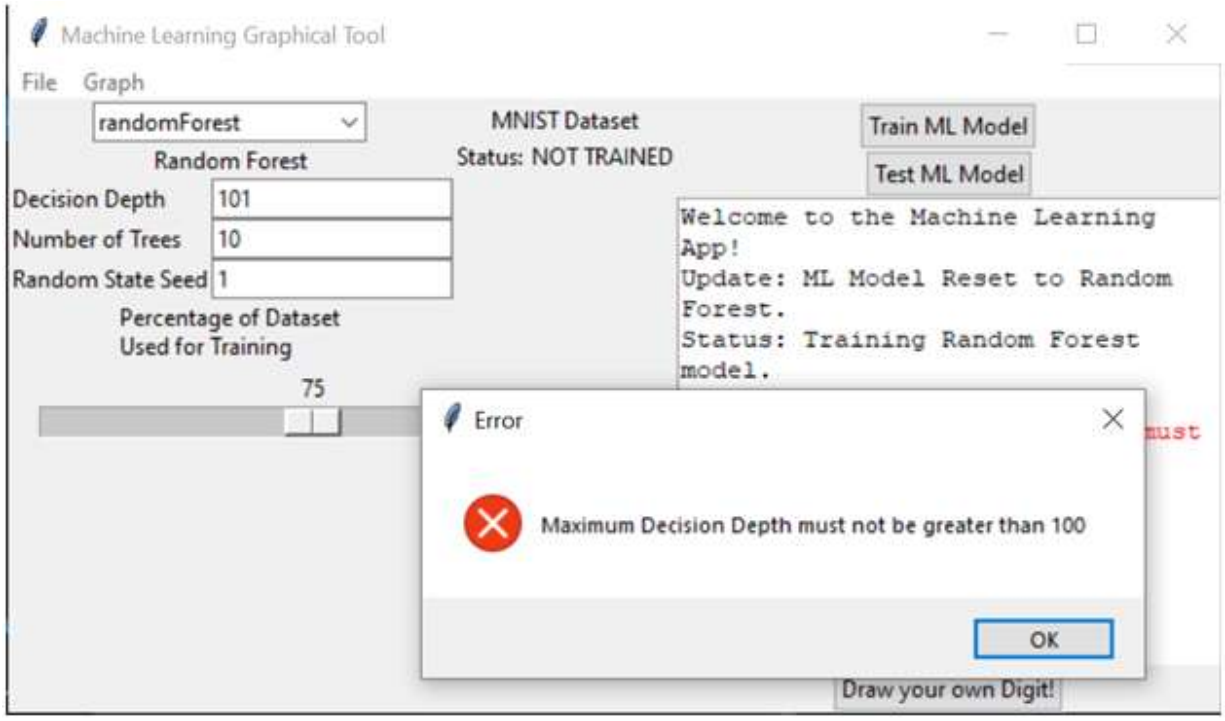
Test Case 32: Inputting "Max Decision Depth" parameter for the Random Forest Learning Model: Zero

Input	Expected Output	Output	Result
Input "0" in the "Max Decision Depth" text box	Error message, noting that a positive integer value is required for max decision depth.	Error message, noting that a positive integer value is required for max decision depth.	Pass



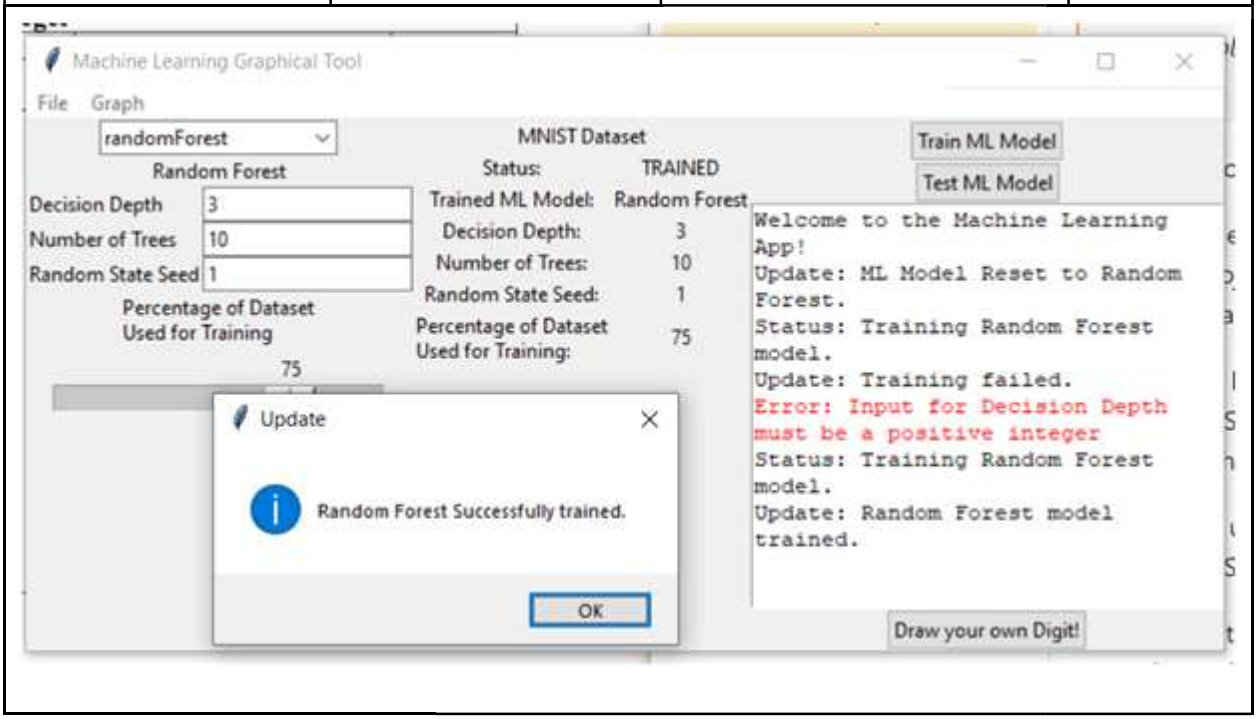
Test Case 33: Inputting "Max Decision Depth" parameter for the Random Forest Learning Model: >= 100

Input	Expected Output	Output	Result
Input "101" in the "Max Decision Depth" text box	Error message, noting that a positive integer value less than or equal to 100 is required for the Max Decision Depth.	Error message, noting that a positive integer value less than or equal to 100 is required for the Max Decision Depth.	Pass



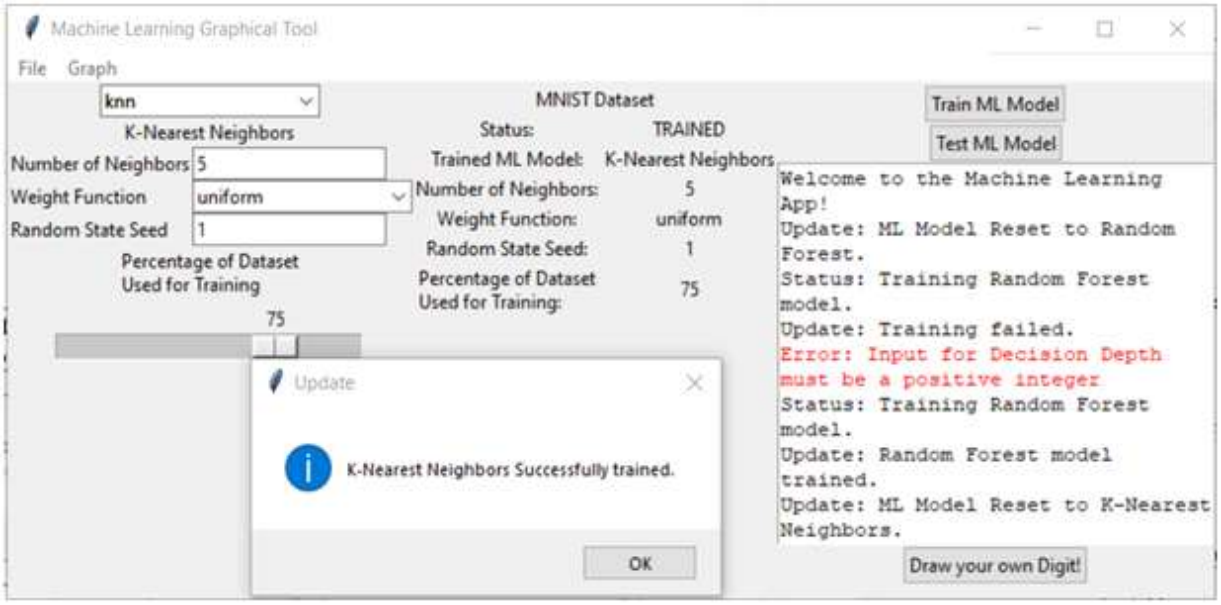
Test Case 34: Inputting "Max Decision Depth" parameter for the Random Forest Learning Model: Positive Integer

Input	Expected Output	Output	Result
Input "3" in the "Max Decision Depth" text box	Random Forest Successfully trained	Random Forest Successfully trained	Pass



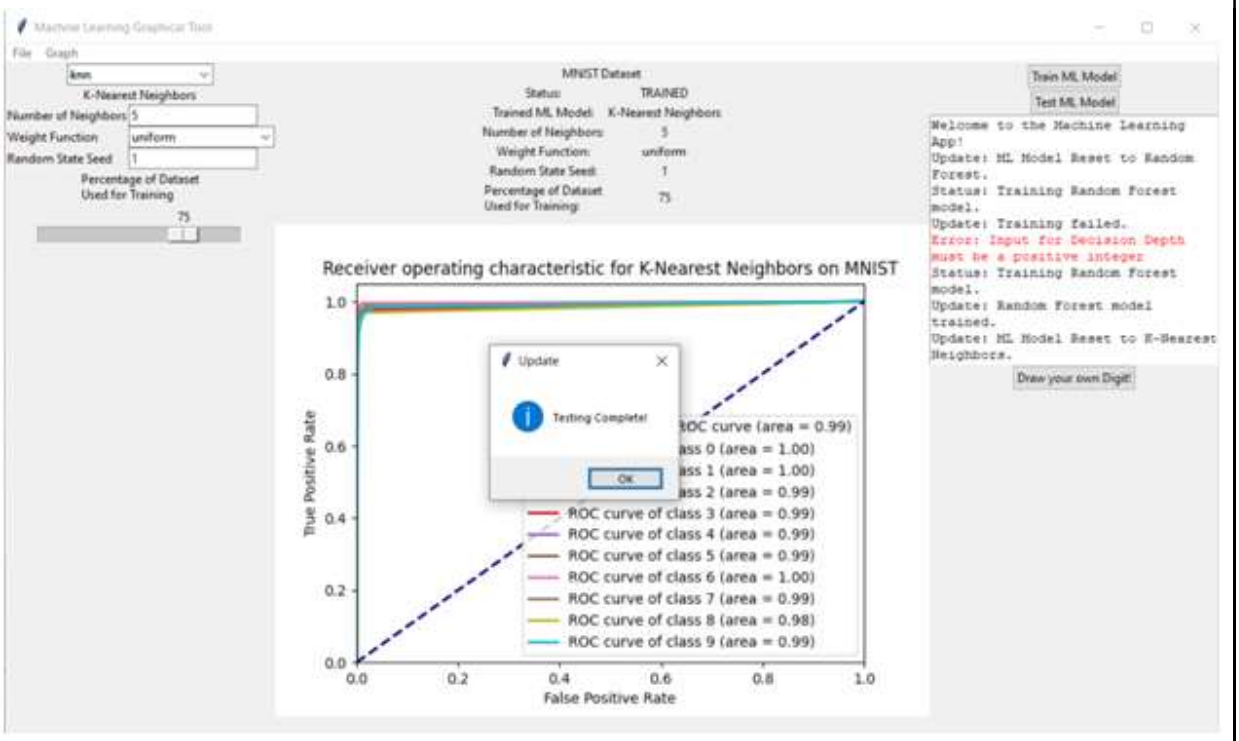
Test Case 35-Part 1: Training K-Nearest Neighbor Machine Learning Algorithm

Input	Expected Output	Output	Result
Press the "Train Machine Model" button	K-Nearest Neighbors Successfully trained	K-Nearest Neighbors Successfully trained	Pass

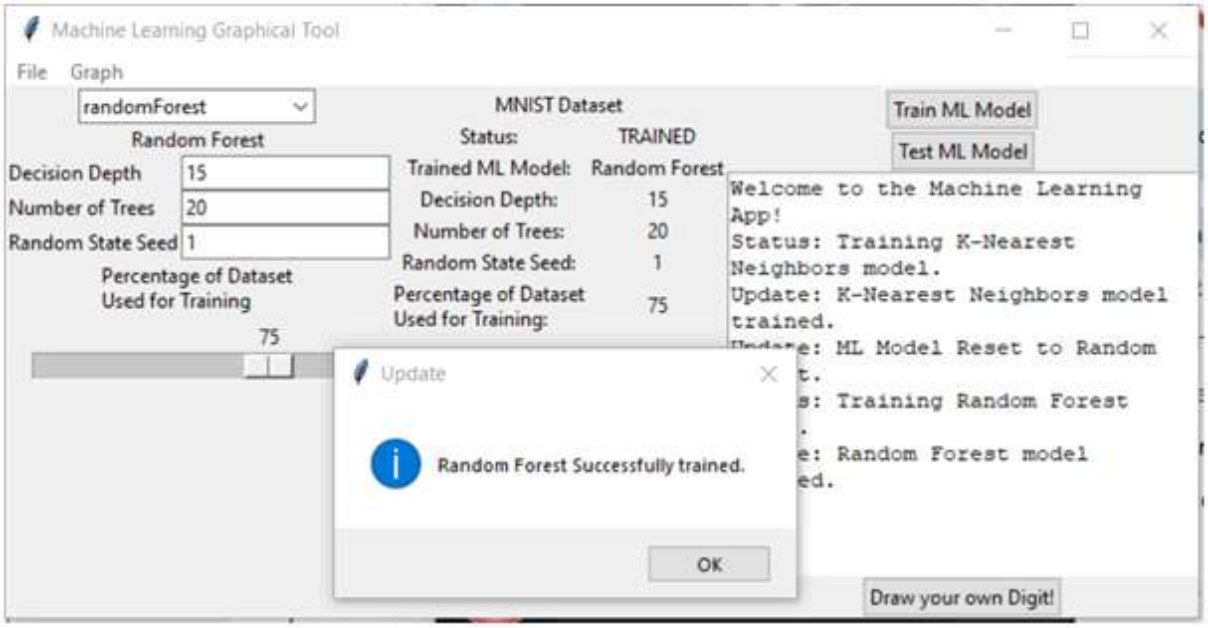


Test Case 35-Part 2: Testing K-Nearest Neighbor Machine Learning Algorithm

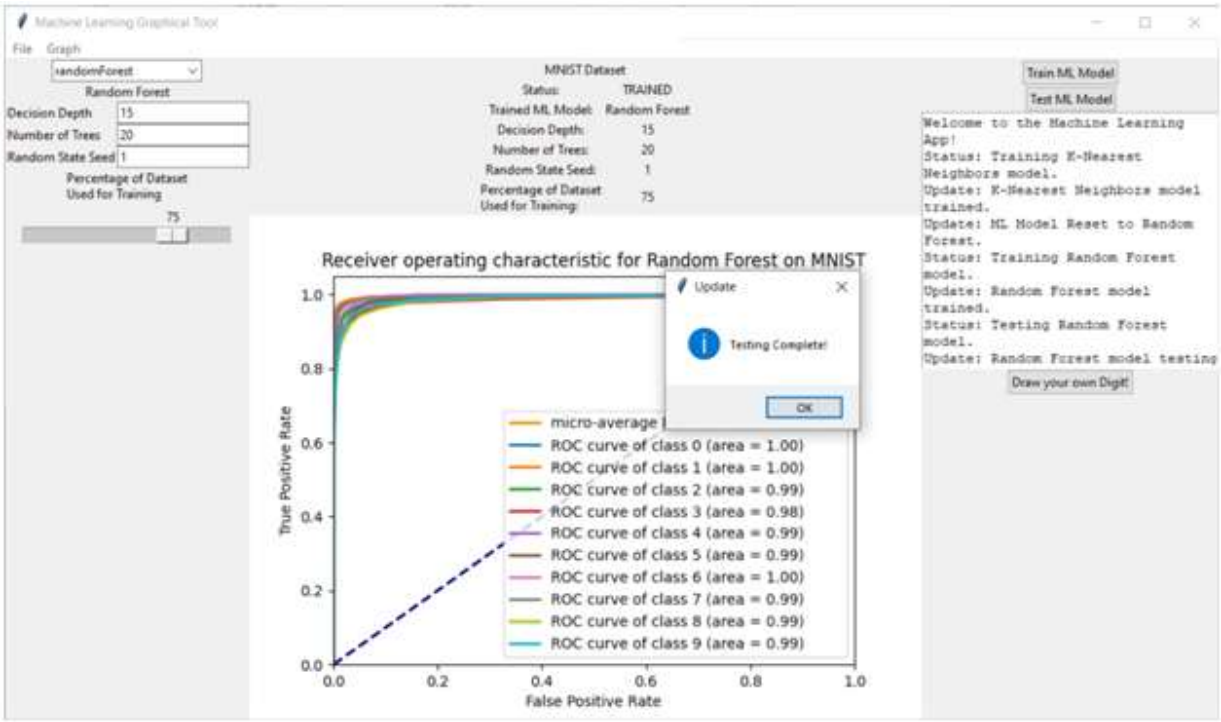
Input	Expected Output	Output	Result
Press the “Test Machine Model” button	Testing Complete	Testing Complete	Pass



Test Case 36: Training Random Forest Machine Learning Algorithm			
Input	**Expected Output**	**Output**	**Result**
Press the "Train Machine Model" button	Random Forest Successfully trained	Random Forest Successfully trained	Pass

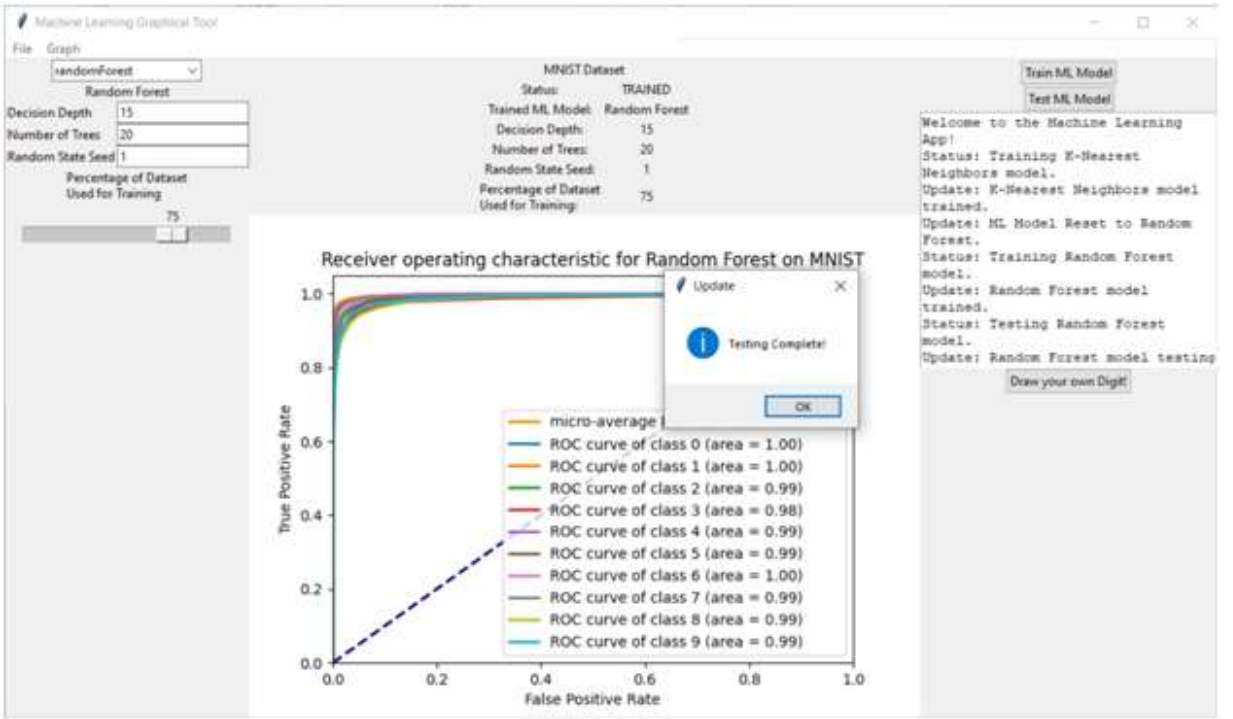


Test Case 36-2: Testing Random Forest Machine Learning Algorithm			
Input	**Expected Output**	**Output**	**Result**
Press the "Test Machine Model" button	Testing Complete	Testing Complete	Pass



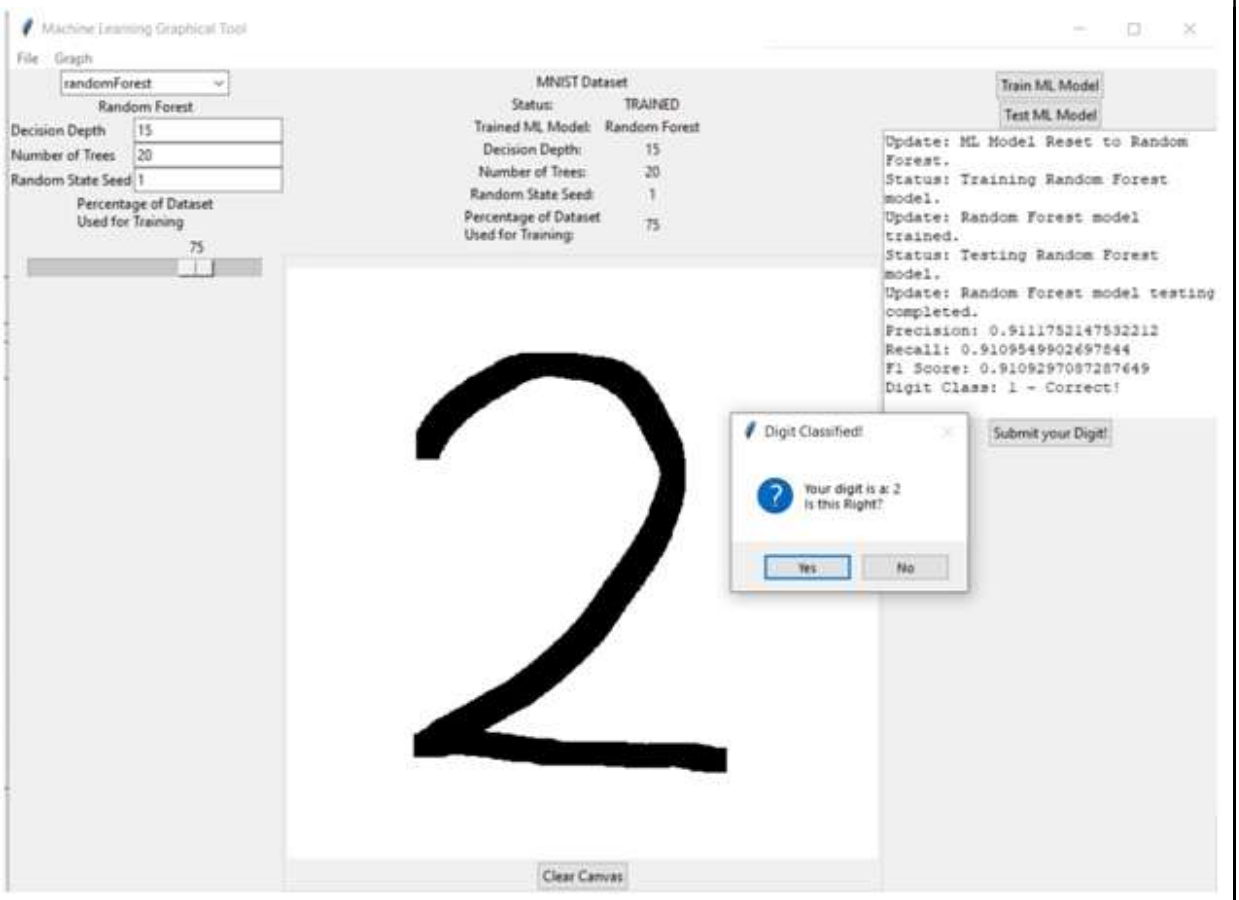
Test Case 36-3: Testing Random Forest Machine Learning Algorithm: Hand Drawn Digits

Input	Expected Output	Output	Result
Draw 1	Classifies as 1	Classified as 1	Pass



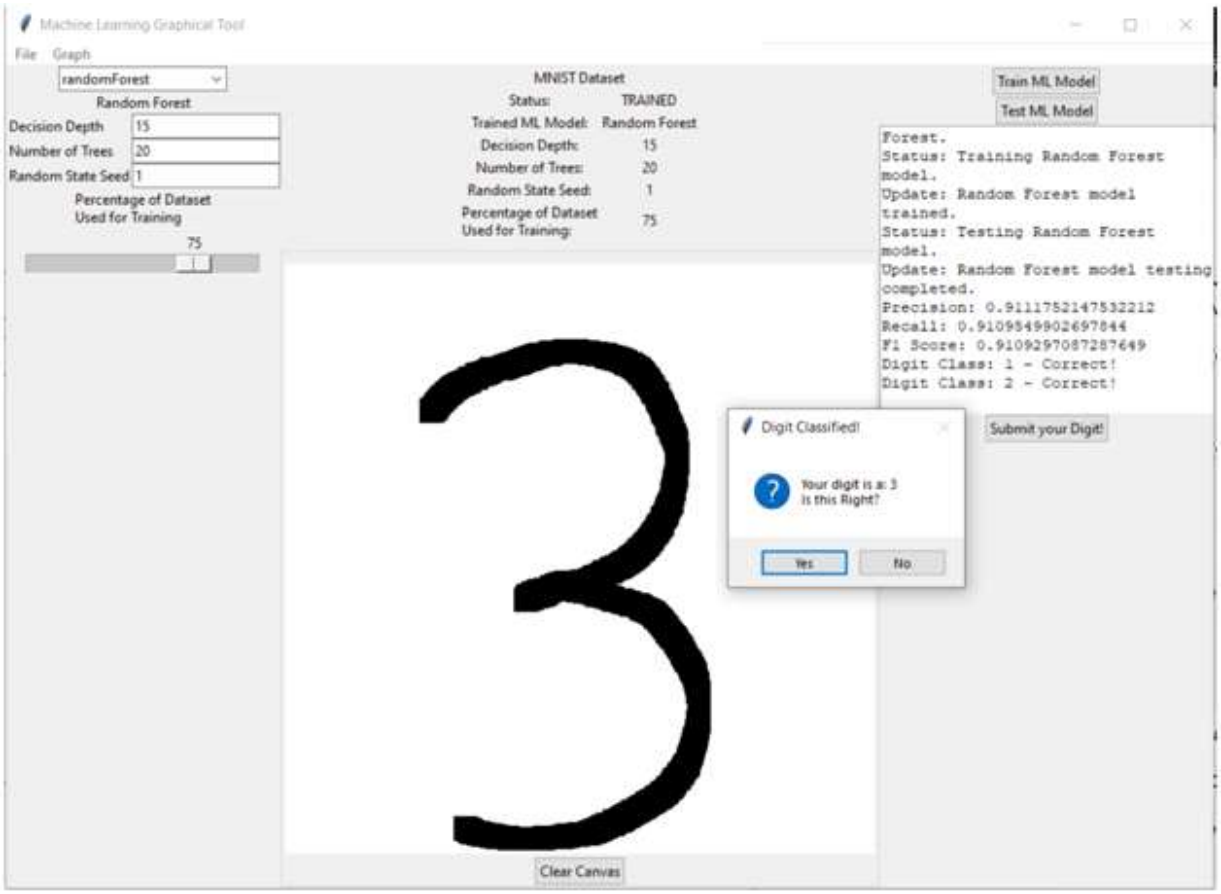
Test Case 36-4: Testing Random Forest Machine Learning Algorithm: Hand Drawn Digits

Input	Expected Output	Output	Result
Draw 2	Classifies as 2	Classified as 2	Pass



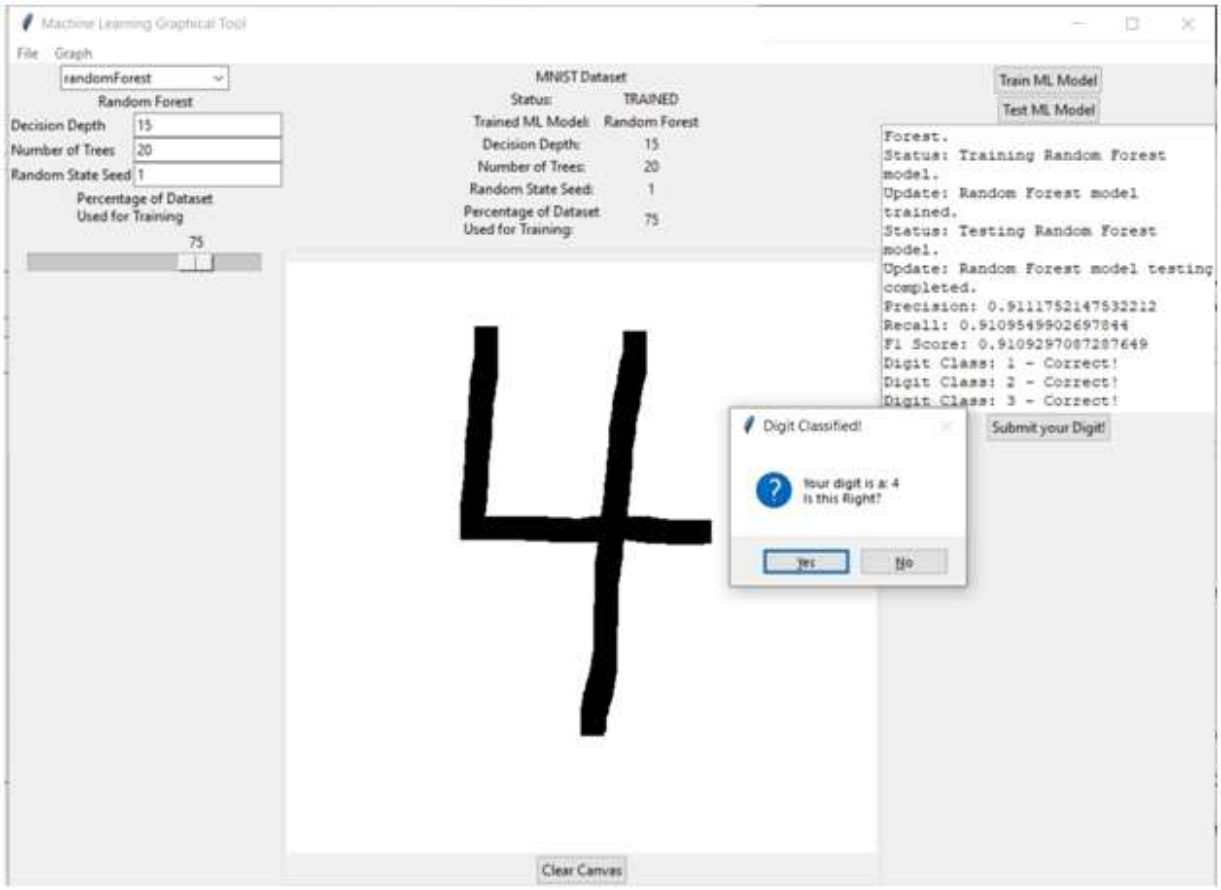
Test Case 36-5: Testing Random Forest Machine Learning Algorithm: Hand Drawn Digits

Input	Expected Output	Output	Result
Draw 3	Classifies as 3	Classified as 3	Pass



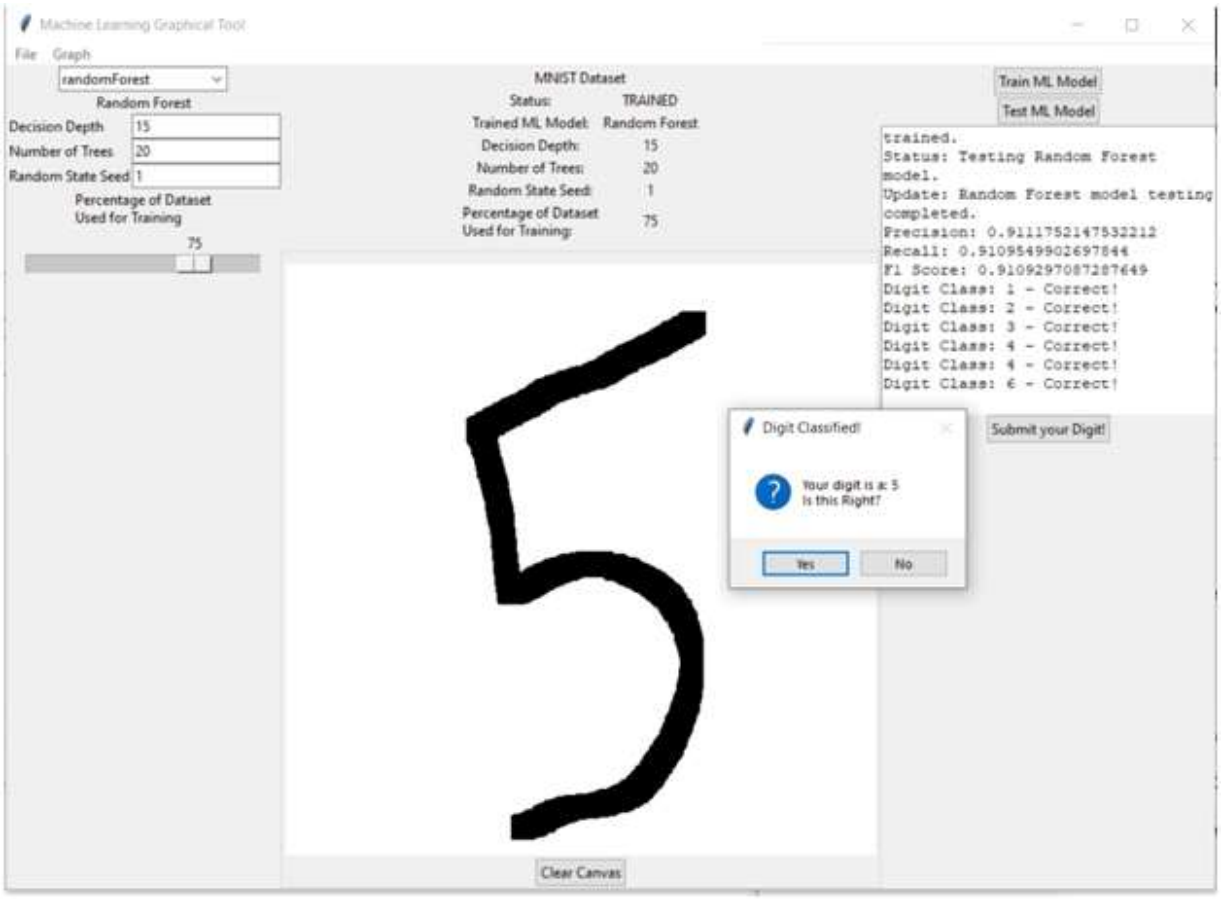
Test Case 36-6: Testing Random Forest Machine Learning Algorithm: Hand Drawn Digits

Input	Expected Output	Output	Result
Draw 4	Classifies as 4	Classified as 4	Pass



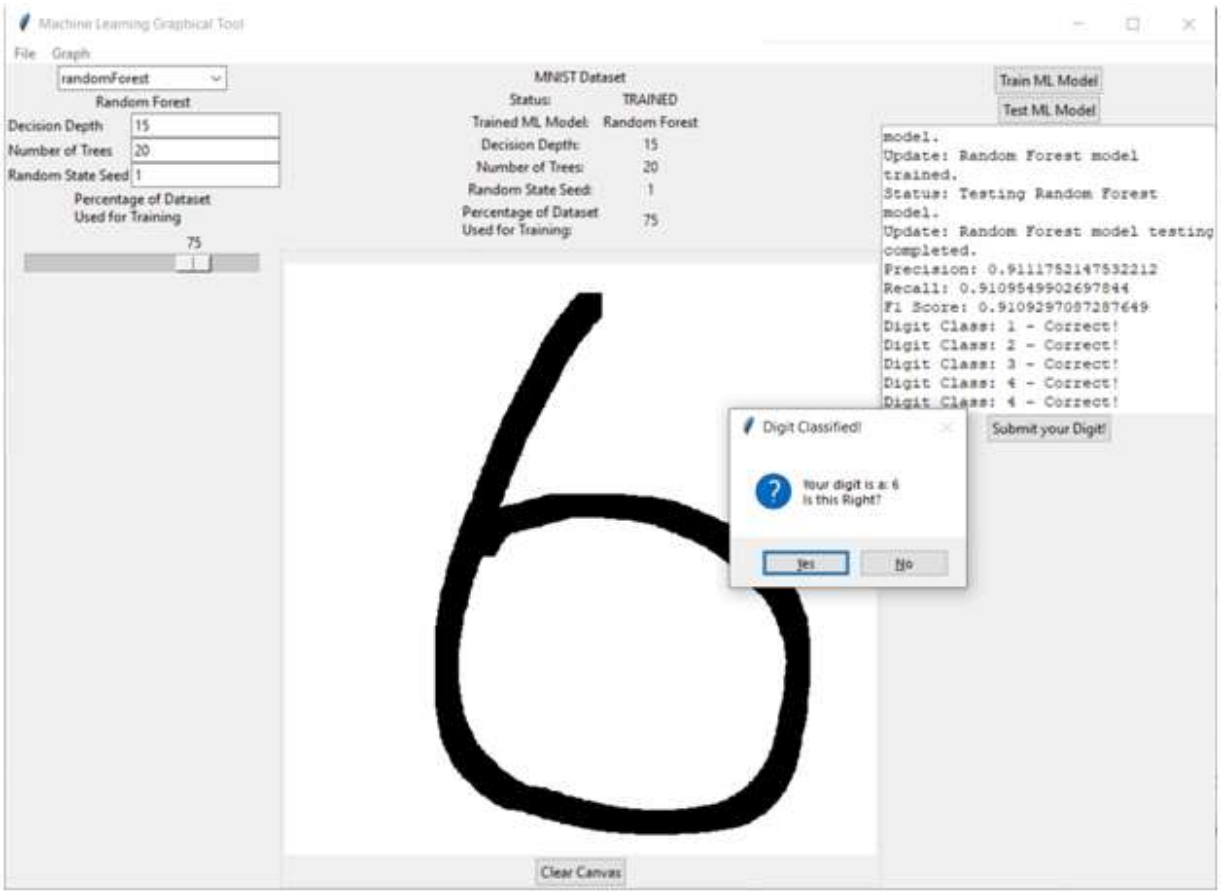
Test Case 36-7: Testing Random Forest Machine Learning Algorithm: Hand Drawn Digits

Input	Expected Output	Output	Result
Draw 5	Classifies as 5	Classified as 5	Pass



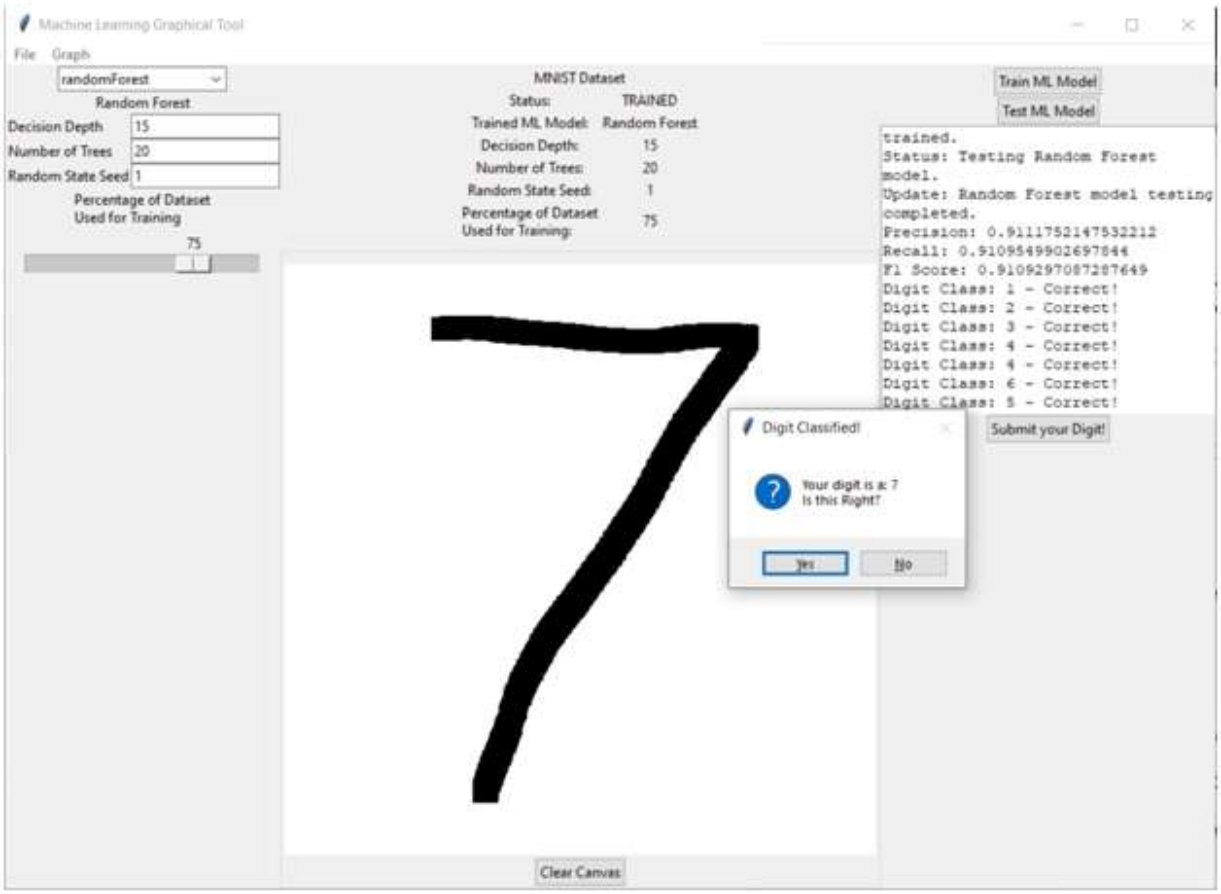
Test Case 36-8: Testing Random Forest Machine Learning Algorithm: Hand Drawn Digits

Input	Expected Output	Output	Result
Draw 6	Classifies as 6	Classified as 6	Pass



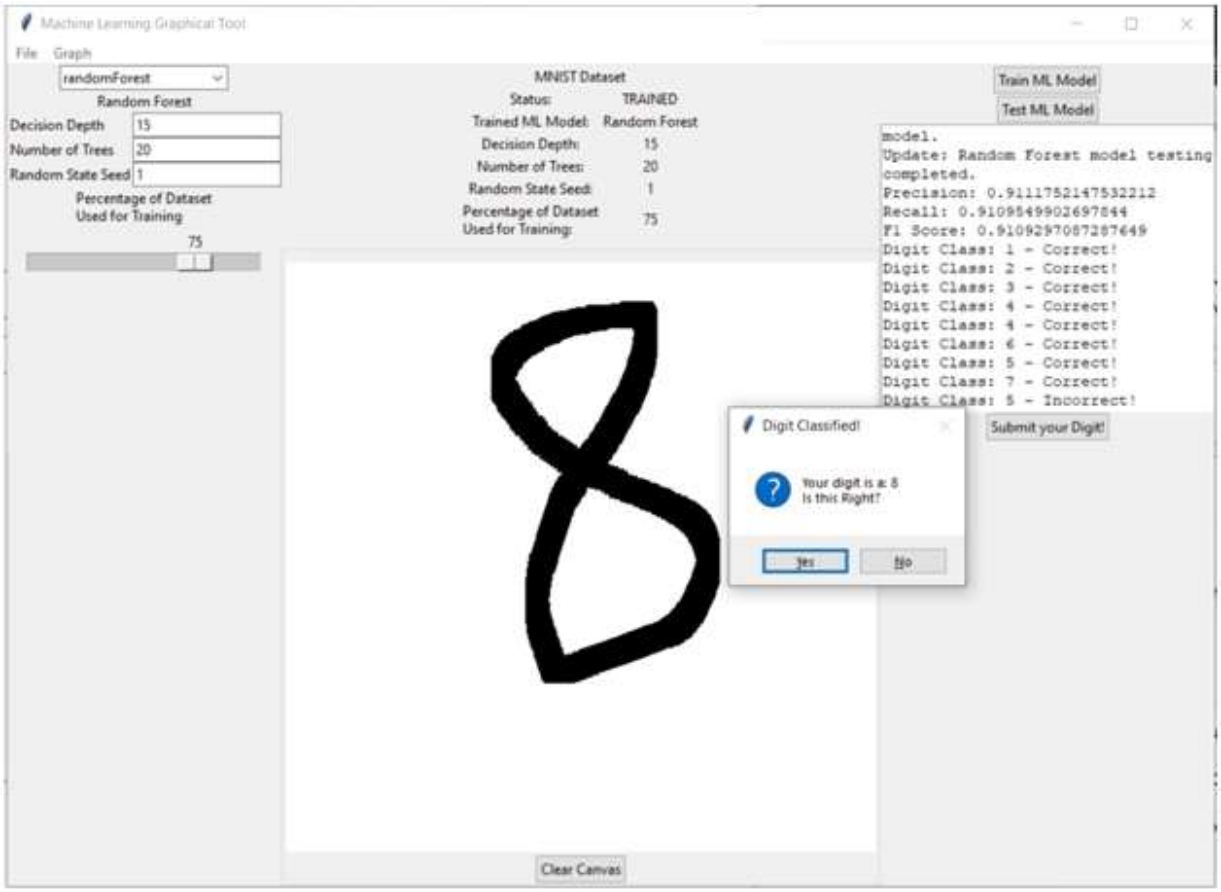
Test Case 36-9: Testing Random Forest Machine Learning Algorithm: Hand Drawn Digits

Input	Expected Output	Output	Result
Draw 7	Classifies as 7	Classified as 7	Pass



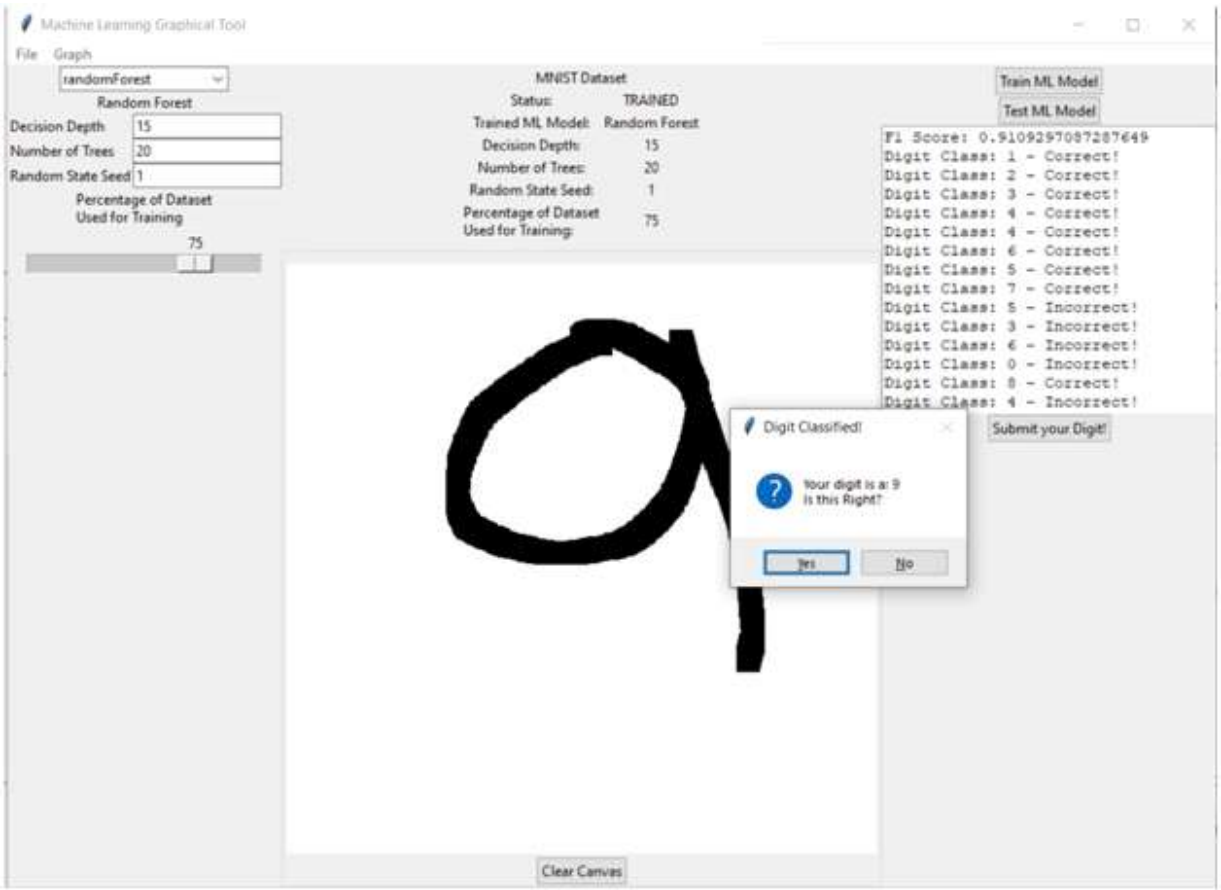
Test Case 36-10: Testing Random Forest Machine Learning Algorithm: Hand Drawn Digits

Input	Expected Output	Output	Result
Draw 8	Classifies as 8	Classified as 8	Pass



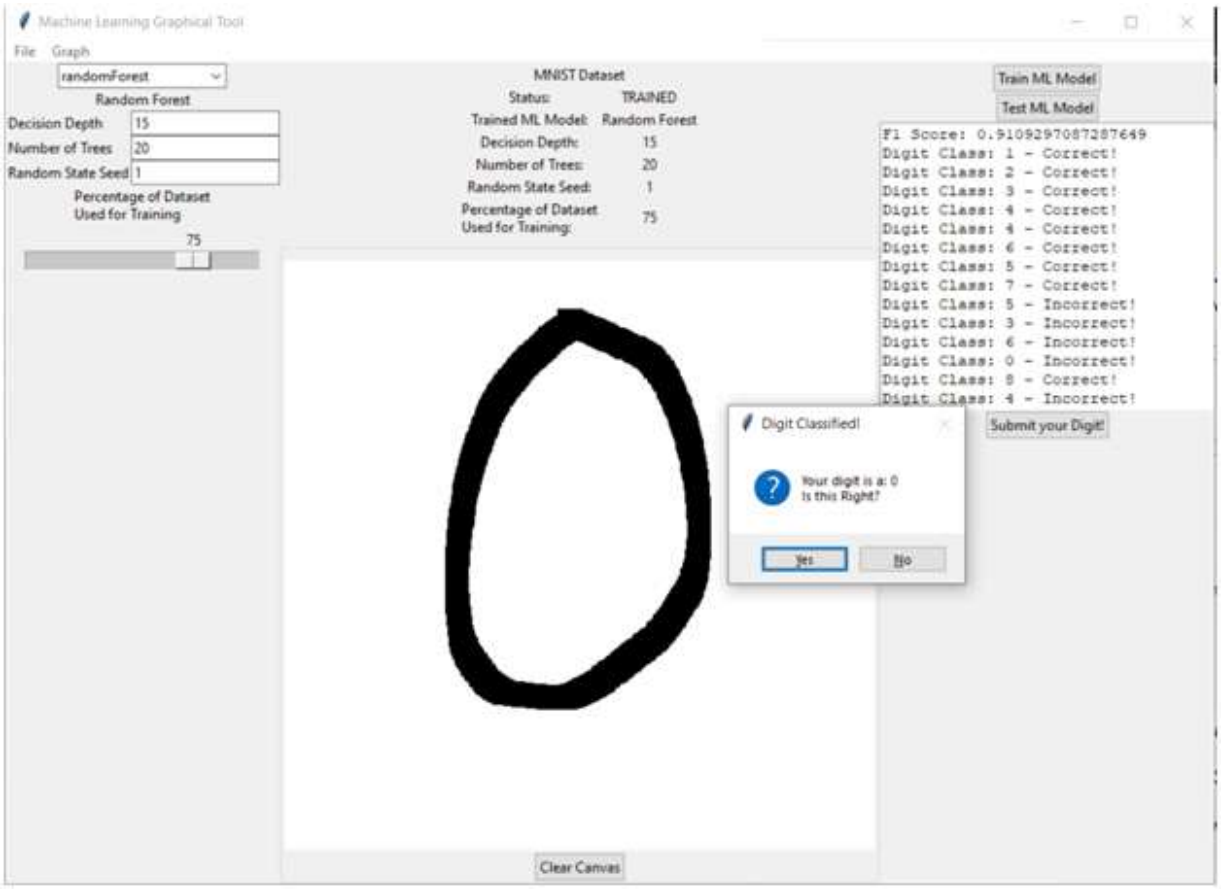
Test Case 36-11: Testing Random Forest Machine Learning Algorithm: Hand Drawn Digits

Input	Expected Output	Output	Result
Draw 9	Classifies as 9	Classified as 9	Pass



Test Case 36-12: Testing Random Forest Machine Learning Algorithm: Hand Drawn Digits

Input	Expected Output	Output	Result
Draw 0	Classifies as 0	Classified as 0	Pass



Design and Alternate designs

Graphical User Interface:

The GUI interface was designed and implemented using the TKinter library in Python. The outermost GUI frame class, ML_GUI.py, represents the executable code for the GUI interface. This class was designed to call the methods of the ML_Function class, which served as a layer of abstraction between the ML_GUI.py class and the Machine Learning model classes. This design allowed for the development of the GUI layout and functionality to proceed simultaneously and simplified the ML_GUI.py code.

The second class, ParameterReader.py, was designed to allow for user input of machine learning model parameters. This class created interactive TKinter widgets, such as text entry boxes, combo boxes, check-boxes, and scales or sliders, based on a limited hard-coded definition of parameter types. This class was designed to handle parameter input for all machine-learning models.

The third class, DigitCanvas.py, allowed the user to draw a black-and-white image directly onto a canvas for conversion into an array compatible with the MNIST dataset. This class relied on a central TKinter Canvas widget for drawing and modifying the elements of a binary integer array parallel to the Canvas. This matrix was then passed to the ImgMatConv class for transformation into an MNIST-compatible array.

ImgMatConv.py, was originally for transforming a black-and-white image file into a binary integer matrix, which was then further processed to transform the data to a form compatible with the data in the MNIST dataset. This class was refined over time to improve its accuracy and performance.

Finally, additional functionality classes, such as GraphFrame.py, were developed to display matplotlib charts directly into the GUI. The format for reading in the required parameters was moved from outside the ParameterReader.py file to a Dictionary data structure in its own file (ML_Model_Params.py) to allow for simpler modification of parameters per model.

The K-Nearest Neighbors Algorithm:

The original implementation of the K-Nearest Neighbors algorithm inherits from the sklearn.base.BaseEstimator class, which allows users to get and set the parameters of the estimator as a dictionary.

The class has four instance variables: k, n_trees, n_jobs, X_train, y_train, and index. The k variable represents the number of nearest neighbors to consider when making a prediction, n_trees is the number of trees to use when building the AnnoyIndex data structure, n_jobs is the number of parallel jobs to run, and X_train and y_train represent the training data.

The class has three methods: fit(), predict(), and predict_proba().

The fit() method takes as input a pd.Dataframe of features X_train and a pd.Series of target y_train, and builds an AnnoyIndex data structure using the X_train data. Each data point in X_train is added to the AnnoyIndex object using its index as a unique identifier. The fit() method then builds the AnnoyIndex object with the specified number of trees (n_trees).

The predict() method takes as input a pd.Dataframe of features X_test and returns a numpy array of predicted labels for each test sample in X_test. For each data point in X_test, the get_nns_by_vector method is used to retrieve the k nearest neighbors of the query point in the training dataset. The method then finds the most common label among the k nearest neighbors using the max() and key() functions.

The predict_proba() method takes as input a pd.Dataframe of features X_test and returns a numpy array of shape (n_samples, n_classes) with predicted probabilities. For each data point in X_test, the get_nns_by_vector method is used to retrieve the k nearest neighbors of the query point in the training dataset. The method then computes the probability of each class using the proportion of neighbors with the same label over the total number of neighbors.

The Random Forest Algorithm:

The original implementation of the Random Forest Algorithm takes in two parameters, the number of trees and the maximum depth of each tree. The constructor initializes these parameters along with X_train, y_train, and trees as class attributes.

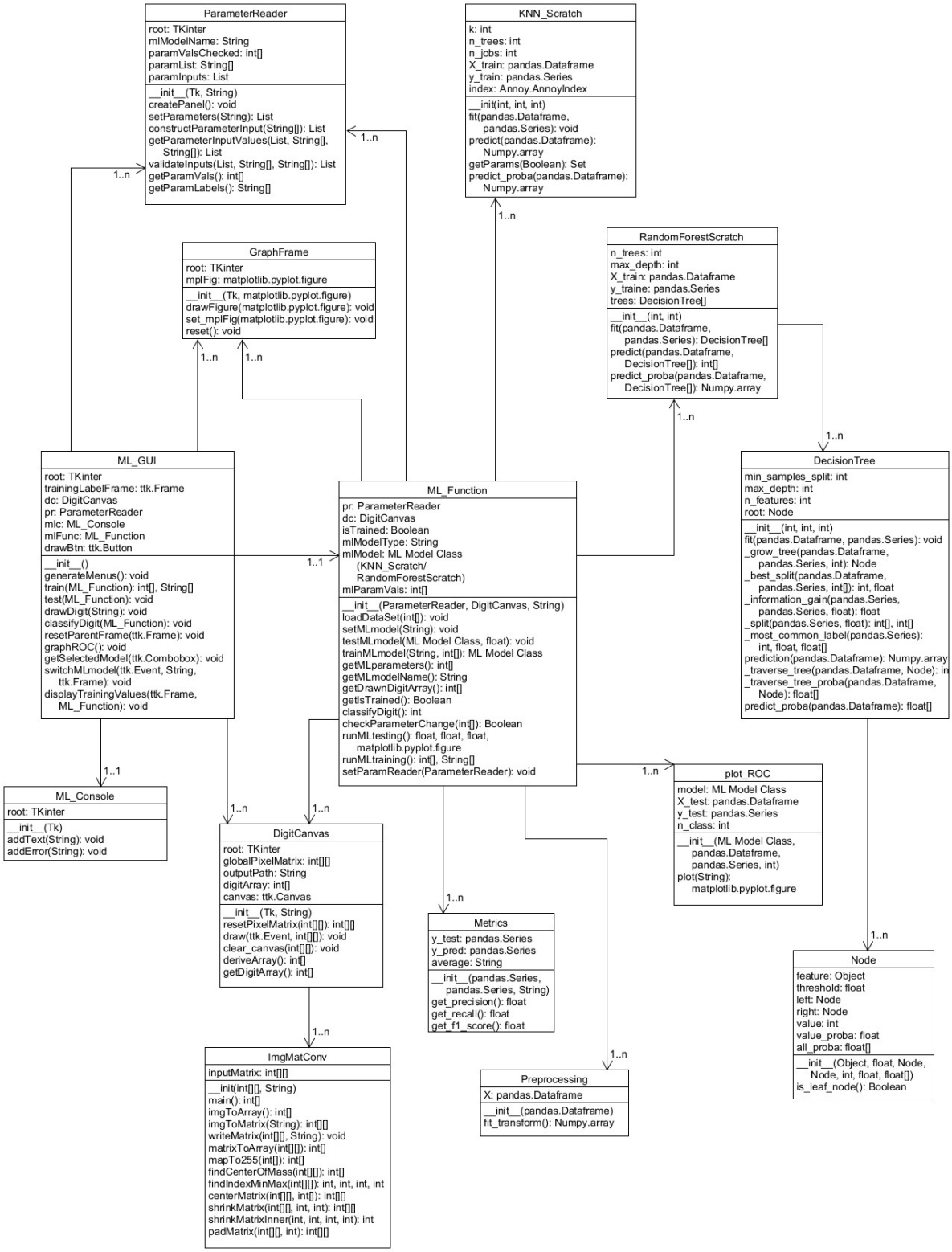
The fit() method takes training data as input and splits the data into the number of trees specified. It then trains a decision tree on each split of the data and stores the trees in a list. Finally, it returns the list of trees.

The predict() method takes a test set as input and uses the list of trees generated from the fit() method to predict the class labels for each data point. It then tallies the votes of each tree prediction and returns the class labels with the highest number of votes.

The predict_proba() method returns an array of probabilities for each class, representing the average of the probabilities among all trees. It first loops over each tree in the list and calls the predict_proba() method of the DecisionTree class, which returns an array of probabilities for each class. It then calculates the mean of these probabilities across all trees and returns the result.

The class also includes additional methods to handle compatibility with a graphical user interface (GUI) such as allowing the user to input a forest for predict_proba() method.

UML Diagram of the MNIST APP Design: (Also available as a PDF file in the submission archive)



Development History

The MNIST App went through several distinct phases of development, each of which is described below.

Planning

The MNIST App was planned to accomplish three major goals:

1. Write original implementations of the K-Nearest Neighbors and Random Forest machine learning models.

2. Write a GUI program that could then train these models on the MNIST dataset, and test their results.

3. Include a feature in the GUI program that would allow the user to draw a single-digit number for classification by these machine learning models – in effect, to extend these models' classification of digits to the user's own input.

Planning was carried further with the specification of requirements (particularly for the GUI and its functions) and the development of preliminary GUI mockups and UML diagrams for a projected program. The program proved much more complex than the initial UML design envisioned.

After planning, programming went through four distinct phases.

Phase 1

GUI programming began with the development of a class (ImgMatConv.py) that could transform a black-and-white image file into an integer matrix, and perform transformations (such as centering the matrix along its center of mass and reducing its resolution) necessary to make the data in that matrix compatible with the data in the MNIST dataset, which consists of approximately 70,000 arrays (rows) each 784 elements long, with each element valued as an integer between 0 and 255 – that is to say, a grayscale pixel.

As this was ongoing, a class for inputting machine learning model parameters (ParameterReader.py) was also developed. This class needed to create interactive TKinter "widgets" allowing for user input, with the widget types corresponding to the kind of input required (for example, a text entry box for integer values, a combo box for limited selections, a check-box for binary choices, and a scale or slider for a limited range of values). This class was designed so that it could automatically generate the needed input widgets based on a limited hard-coded definition of parameter types. The major benefit to this design is that it allowed one class to handle parameter input for all machine-learning models.

The design and development of the outermost GUI frame class (represented as the executable ML_GUI.py class) began at this time, as well.

As for the implementation of the K-Nearest Neighbors algorithm, the developer started by building a baseline model for the K-Nearest Neighbors algorithm. This initial implementation used an exhaustive search over all training samples to find the k nearest neighbors for each test example.

However, due to the high dimensionality of the feature space (28x28=784) in the MNIST dataset, this approach was inefficient. To overcome this challenge, the developer proposed several possible improvements, including using kd-trees or ball trees, reducing dataset dimensionality, using approximate nearest neighbors, and employing parallel processing.

With that in mind, the developer proceeded to develop the second version of the implementation with the below modifications:

- Utilizing NumPy broadcasting to compute pairwise distances between test and training data.
- Using np.argpartition to obtain indices of the k-nearest neighbors more efficiently.
- Using collections.Counter to count the occurrence of each label among the k-nearest neighbors instead of computing it in a loop.

As for the implementation of the Random Forest algorithm, the developer began with data analysis of the MNIST hand drawn digit dataset. This included experimenting with other random forest models such as the built-in sci-kit learn model as well as singular decision tree models to better understand the scope and process. The next step required writing up the decision tree and forest algorithms.

Phase 2

GUI Programming continued with the development of a class that allowed the user to draw a black-and-white image directly onto a canvas for conversion into an array compatible with the MNIST dataset (DigitCanvas.py). This class relied on a central TKinter Canvas widget for drawing; however, since it is not (to the knowledge of these developers) possible to extract pixel information from a TKinter canvas, it became necessary to include a method that would simultaneously modify the elements of a binary integer array parallel to and of the same dimensions as the Canvas. This matrix would then be passed to the ImgMatConv class for transformation into an MNIST-compatible array.

At the same time, the ImgMatConv class was refined to further improve its accuracy and performance, particularly with centering and resizing the input matrix.

The design of the GUI was concluded at this time, and the development of the ML_GUI class continued.

For the K-Nearest Neighbors algorithm, the developer used scipy.spatial.cKDTree to find the k-nearest neighbors for each point in X_test. This approach was more efficient than computing distances between each test instance and all training instances, as in the previous implementation. Additionally, the developer replaced the use of

numpy.array([Counter(labels).most.common(1)[0][0]) with numpy.argmax(numpy.bincount(labels)). The bincount function computed the frequency of each label in the label array and returned an array where the ith element represented the frequency of

the ith label. The argmax function then found the index of the highest frequency label in this array. In contrast, Counter created a dictionary-like object that stored the frequency of each label in the label array. The most_common(1) method of the Counter object returned a list of tuples, with each tuple containing a label and its frequency. The [0][0] indexing then returned the label with the highest frequency. This change improved the efficiency of the implementation. As the results did not meet the expectation, the developer went on the development of the fourth version, which is also the final version of the implementation. The final version uses Approximate Nearest Neighbors, which split the data into partitions using hyperplanes and traversed the trees searching for approximate nearest neighbors. This approach was more efficient than computing distances between each data point, as in the initial implementation, because it only calculated the distances between the query point and the subset of the candidate's nearest neighbors, rather than computing the distances for all the data points. This resulted in a significant reduction in the number of distance calculations required.

The Random Forest algorithm development continued with the creation of the decision tree. After the successful generation of a tree, development shifted towards reducing the amount of time it takes to grow the tree. As it stood, the growing or fitting of the tree would take anywhere between <1min to 10min, and with consideration to a forest of trees, this time commitment was not ideal. So with the assistance from the group, data compression/decomposition was implemented which reduced the run time by roughly 30-40%. The next venture will be to apply the concept a random forest using the decision tree.

Phase 3

A layer of abstraction was added between the outermost GUI Frame (ML_GUI.py) and the other classes via the development of the ML_Function class. This class was necessary so that the development of the GUI layout and GUI functionality could proceed simultaneously. It also simplified the ML_GUI.py code considerably, as that class could simply call the methods of the ML_Function class, rather than methods from nine other classes.

Additional functionality classes such as for displaying MatPlotLib charts directly into the GUI (GraphFrame.py) were developed at this time as well. The way in which ParameterReader.py was instantiated was changed at this time; the format for “reading in” the required parameters was moved from outside that file, to a Dictionary data structure in its own file (ML_Model_Params.py). This allowed for simpler modification of parameters per model (necessary for adding a parameter for pre-processing the dataset, a feature added during this phase).

The GUI design and implementation were concluded during this time. It implemented training and testing for each of the machine learning models, as well as displaying a Receiver Operator Characteristic chart for each tested model. Inputting parameters and drawing and classifying a single-digit number was already functional.

Model evaluation methods and graphs were also developed in this phase. We provide two classes to evaluate machine learning models and visualize their performance using Sci-kit Learn's methods.

The first class, Metrics, take in the true labels of the test data (y_test) and the predicted labels (y_pred) and computes three evaluation metrics - precision, recall, and F1 score - using precision_score, recall_score, and f1_score functions from Sci-kit Learn. The second class, plot_ROC, takes in a machine learning model (model), test data (X_test and y_test), and the number of classes (n_class) and generates a ROC curve for the model on the test data using roc_curve and auc functions from Sci-kit Learn. The plot method plots the ROC curve for each class, the micro-average ROC curve, and the area under each curve (roc_auc) using Matplotlib. The method also takes an optional modelName parameter, which specifies the name of the model being plotted. It returns a figure object for further customization or use.

In the last stage of development for the random forest classifier, development focused on incorporating the decision tree to generate the desired number of trees with each tree being unique in it's decision nodes. Then ensuring functionality to tally each tree's vote on data features and then finally classifying the datapoint based on the tally. The next development effort was to re-write it as a class and tweak it so that it would be compatible with the GUI. Then we move into the final phase of executing the test plan and implementing fixes.

Phase 4: Finalization

In the final development phase of the project, there are several critical activities that were performed to ensure a smooth and successful deployment of the application. One of the primary activities was the integration of each machine learning algorithm, evaluation method, and graphical user interface. During this phase, the group members worked together to integrate these components seamlessly to ensure the application operates as intended.

To ensure the application meets the project's goals, the group has also developed a test plan that outlines the various tests that need to be conducted. These tests verified that the application functions correctly and is free of bugs. During the testing phase, the group worked diligently to address any issues that arose, ensuring that the application operates smoothly and meets the project's requirements.

Apart from the testing phase, another critical activity during this phase was the development of the complete documentation of the application's development, purpose, and user guide. The documentation provides users with a detailed understanding of the application, its purpose, and how to use it. The user guide provides step-by-step instructions on how to use the application, including information on how to input data, choose algorithms, customize parameters, and interpret the results.

Conclusion

Lessons Learned

While the members of Group 1 had varying levels of experience with programming and project development upon beginning this project, it is not untrue to say that each of us learned new things and expanded our experience during this project. We had to be adaptable and open minded to the potential challenges we faced while implementing this project.

Machine Learning

As the subject of this project, machine learning was a topic that each member had to review and familiarize himself or herself with. As such, each of the Group 1 members improved his or her understanding of machine learning during the development of the MNIST App. In particular, the group members became familiar with two particular machine learning models – the K-Nearest Neighbors and the Random Forest models. The group wrote original implementations of each of these models, which required studying them in depth to understand how they worked, and how they could work better. In each case, improvements were made over time to improve the performance of each model.

Performance optimization is another important aspect that can be learned from implementing algorithms from scratch. By developing one's own implementation, it becomes necessary to optimize the algorithm for better performance, which can involve techniques such as pre-sorting data, minimizing memory usage, and reducing computational overhead. These optimization techniques can help to improve the algorithm's efficiency and scalability, making it better suited for large-scale and real-world applications.

Python Programming

As the language of choice for this project, each of the group members had to become more familiar with the Python programming language. This ran the gamut from coding using basic object-oriented design principles in Python to enhancing one’s understanding of best practices for coding and documentation. The group members also had the opportunity to familiarize themselves and practice with a number of Python libraries, such as Scikit-Learn, NumPy, SciPy, and TKinter.

GUI Design and Development

GUI design was a central part of this project that each group member examined to a greater or lesser extent. Writing a straightforward, functional, and error-free GUI to implement the machine learning models and their auxiliary functions (such as graphing) required considerable time and effort. Additionally, the limitations of TKinter compared to the Java Swing and AWT libraries became apparent during this project; however, the group was able to implement each of the planned goals, and the GUI was a success.

Project Management

The group was split between members in the Eastern US and Japan; as such, a thirteen-hour time-zone difference existed between group members. This made communication and coordination difficult. “Spitballing” was essentially impossible since all group members were rarely available at the same time. Plans were agreed by consensus based on general points, after which the work was divided among individual team members who had, essentially, total freedom to implement their parts of the project as each saw fit. There could be no “oversight” per se in this project, so individual initiative was the premium quality. This sort of collaboration resembled an open-source type project and was an informative and enriching experience for each of the group members. It enabled each group member to focus on his or her individual strengths while retaining the potential to receive help or guidance from another group member if requested.

Design Strengths and Limitations

Developing one's own version of an algorithm provides flexibility in prioritizing certain aspects of the implementation. However, achieving the same level of efficiency as established libraries, such as scikit-learn, can be difficult.

Regarding the K-nearest neighbors algorithm implementation:

In this project, the K-nearest neighbors algorithm was developed in four versions. However, the first two versions failed to handle the MNIST dataset due to memory limitations. The third version used KDtree implementation, which efficiently searches for the k-nearest neighbors of a query point by considering regions of the feature space likely to contain the nearest neighbors. Despite this, the third version still took twice as long as scikit-learn. Therefore, an approximate nearest neighbors (ANN) method was used instead of the exact K-nearest neighbor approach, as ANN aims to find an approximate set of nearest neighbors that are "close enough" to the true nearest neighbors to achieve similar performance as scikit-learn's KNeighborClassifier. It is believed that scikit-learn's superior performance is due to optimization techniques, such as pre-sorting the data by feature values and sorting distance calculations in a buffer, some methods being written in C or CPython, and extensive testing and optimization for a wider range of scenarios and datasets.

Regarding the Random Forest algorithm implementation:

The random forest algorithm was developed with various singular decision tree algorithms. A few of these trees did not work with the MNIST dataset and had to be scrapped. Once a working decision tree was developed and incorporated into the random forest algorithm, the focus was on optimizing and reducing run time. However, due to limited experience and time constraints this implementation of a random forest does not come anywhere close to scikit-learn’s RandomForestClassifier in terms of speed and accuracy. In fact the only way to consistently improve accuracy is to increase the number of decision nodes within each tree which is what drives longer run times. While it can achieve accuracy greater than 92%, the trade-off is a significant run time.

Suggestions for Future Improvement

The major improvement which would have been welcome to this project would have been an expansion in the number of machine learning models made available, and the number and types of parameters for each of these models. However, as only one team member had previous experience with machine learning, the group simply was not able to write more models or more complex variants of the implemented models in the limited time allotted for this project, particularly since much of that time had to be devoted to learning about machine learning in the first place. Had the group been larger, had it had more time, or had it had more prior experience with machine learning, no doubt a more robust program with more features would have been developed.

Another, related potential improvement, would have been to undertake more efficient implementations of machine learning models. Possible improvements that can be made to the algorithm implementation:

- Batch processing: Instead of computing predictions for each test example one at a time, we could use batch processing to compute predictions for multiple test examples at once. This could improve the efficiency of the algorithm, especially for large datasets.
- Algorithm modification, for instance for the KNN implementation, weighted voting or using different methods for selecting the value of k. Experimenting with these modifications could improve the performance of the algorithm.

However, this would, once again, have required either more time or more prior experience with machine learning.

A third potential improvement that may have been welcome would have been the implementation of a superior, “draw your own digit” feature, which could have recognized multiple digits in a sequence, or even whether a user drew a figure which was not a number. This may not have been possible with the MNIST dataset; however, the group did not have the time to examine whether this was the case.

Additional, but smaller, features which might have been included in this program, but which were not due to time constraints, were:

- A feature to save machine learning models and parameters to a file for future loading
- Additional metrics for gauging machine learning effectiveness
- Inclusion of additional datasets, with a selection of datasets possible
- Inclusion of a means to use only part of a dataset, rather than the dataset as a whole.

All in all the collaborative effort of Group 1 proved to be a success. We now have experience of how to successfully work with other people to attain a common goal.